

```

510 register_cpu_notifier(&ratelimit_nb);
511 }
-----

```

第 495~507 行: 如果正在操作的机器拥有的页缓存数量相对于缓冲区页面数量来说, 数量众多, 因而可以降低脏页回写的阈值。如果不降低该阈值而提高回写频率, 那么每次回写时就需要使用大量 buffer_heads。(这就是 page_writeback() 前面那个注释的意思。)

默认的后台回写 dirty_background_ratio 从有 10% 的脏页时开始。专门的回写 vm_dirty_ratio 从 40% 的页面为脏页时开始。

第 508 行: 修改回写定时器 wb_timer, 使之被周期性地触发 (在默认状态下, 每 5s 触发一次)。

第 509 行: 调用 set_ratelimit(), 该函数有极详细的文档说明。其内部注释如下。

```

-----
mm/page-writeback.c
450 /*
451 * If ratelimit_pages is too high then we can get into dirty-data overload
452 * if a large number of processes all perform writes at the same time.
453 * If it is too low then SMP machines will call the (expensive)
454 * get_writeback_state too often.
455 *
456 * Here we set ratelimit_pages to a level which ensures that when all CPUs
are
457 * dirtying in parallel, we cannot go more than 3% (1/32) over the dirty
memory
458 * thresholds before writeback cuts in.
459 *
460 * But the limit should not be set too high. Because it also controls the
461 * amount of memory which the balance_dirty_pages() caller has to write back.
462 * If this is too large then the caller will block on the IO queue all the
463 * time. So limit it to four megabytes - the balance_dirty_pages() caller
464 * will write six megabyte chunks, max.
465 */
466
467 static void set_ratelimit(void)
468 {
469     ratelimit_pages = total_pages / (num_online_cpus() * 32);
470     if (ratelimit_pages < 16)
471         ratelimit_pages = 16;
472     if (ratelimit_pages * PAGE_CACHE_SIZE > 4096 * 1024)
473         ratelimit_pages = (4096 * 1024) / PAGE_CACHE_SIZE;
474 }
-----

```

第 510 行: page_writeback_init() 最后的命令向 CPU 通告程序注册名为 ratelimit_nb 的速率限制通告程序块。被通告时, 这个通告程序块调用 ratelimit_handler(), 该函数再来调用 set_ratelimit(), 其目的在于, 当在线 CPU 的数目发生变化时, 重新计算 ratelimit_pages。

```

-----
mm/page-writeback.c

```

```

483 static struct notifier_block ratelimit_nb = {
484     .notifier_call = ratelimit_handler,
485     .next = NULL,
486 };

```

最后, 还需要检查当 wb_timer (从第 508 行开始) 离开并调用 wb_time_fn() 时究竟发生了什么:

```

-----
mm/page-writeback.c
414 static void wb_timer_fn(unsigned long unused)
415 {
416     if (pdflush_operation(wb_kupdate, 0) < 0)
417         mod_timer(&wb_timer, jiffies + HZ); /* delay 1 second */
418 }
-----

```

第416~417行: 当定时器响起时, 内核触发 pdflush_operation(), 该函数唤醒一个 pdflush 线程将实际的脏页写回磁盘。如果 pdflush_operation() 不能唤醒任何 pdflush 线程, 那么它将通知回写定时器在 1s 内再次触发, 试图唤醒一个 pdflush 线程。有关 pdflush 的详情参见第9章。

8.5.30 调用 proc_root_init()

第 480~482 行: 在第 2 章中, 我们已经解释过 CONFIG_#define 是指编译时的变量。若在编译时选中了 proc 文件系统, 初始化的下一步操作就是调用 proc_root_init()。

```

-----
fs/proc/root.c
40 void __init proc_root_init(void)
41 {
42     int err = proc_init_inodecache();
43     if (err)
44         return;
45     err = register_filesystem(&proc_fs_type);
46     if (err)
47         return;
48     proc_mnt = kern_mount(&proc_fs_type);
49     err = PTR_ERR(proc_mnt);
50     if (IS_ERR(proc_mnt)) {
51         unregister_filesystem(&proc_fs_type);
52         return;
53     }
54     proc_misc_init();
55     proc_net = proc_mkdir("net", 0);
56 #ifdef CONFIG_SYSVIPC
57     proc_mkdir("sysvipc", 0);
58 #endif
59 #ifdef CONFIG_SYSCTL
60     proc_sys_root = proc_mkdir("sys", 0);
61 #endif

```

```

62  #if defined(CONFIG_BINFMT_MISC) || defined(CONFIG_BINFMT_MISC_MODULE)
63      proc_mkdir("sys/fs", 0);
64      proc_mkdir("sys/fs/binfmt_misc", 0);
65  #endif
66      proc_root_fs = proc_mkdir("fs", 0);
67      proc_root_driver = proc_mkdir("driver", 0);
68      proc_mkdir("fs/nfsd", 0); /* somewhere for the nfsd filesystem to be
        mounted */
69  #if defined(CONFIG_SUN_OPENPROMFS) || defined(CONFIG_SUN_OPENPROMFS_MODULE)
70      /* just give it a mountpoint */
71      proc_mkdir("openprom", 0);
72  #endif
73      proc_tty_init();
74  #ifdef CONFIG_PROC_DEVICETREE
75      proc_device_tree_init();
76  #endif
77      proc_bus = proc_mkdir("bus", 0);
78  }

```

第 42 行: 本行初始化为索引节点缓存, 用于保存该文件系统的索引节点。

第 45 行: 向内核注册 `file_system_type` 类型的结构 `proc_fs_type`。我们来认真分析这一结构。

```

-----
fs/proc/root.c
33  static struct file_system_type proc_fs_type = {
34      .name      = "proc",
35      .get_sb     = proc_get_sb,
36      .kill_sb    = kill_anon_super,
37  };

```

`file_system_type` 结构简单地将文件系统的名字定义为 `proc`, 该结构包含检索和释放超级块结构的例程。

第 48 行: 挂载文件系统 `proc`。欲知此处发生了什么, 详情可查阅 `kern_mount` 的附加内容。

第 54~78 行: 调用 `proc_misc_init()` 来创建 `/proc` 文件系统中绝大多数的项。它通过调用 `create_proc_read_entry()`、`create_proc_entry()` 和 `create_proc_seq_entry()` 来创建项。该代码块的剩余部分由 `proc_mkdir` 调用和 `proc_tty_init()` 例程调用组成, 它们分别用于创建 `/proc/` 下的目录和 `/proc/tty` 下的树。如果设置了 `CONFIG_PROC_DEVICETREE` 的配置时间值, 那么, 还要调用 `proc_device_tree_init()` 例程来创建 `/proc/device-tree` 子树。

8.5.31 调用 `init_idle()`

第 490 行: `start_kernel()` 快结束时使用参数 `current` 来调用 `init_idle()`, 而 `smp_processor_id()` 则准备重新调度 `start_kernel()`。

```

-----
kernel/sched.c
2643 void __init init_idle(task_t *idle, int cpu)

```

```

2644 {
2645     runqueue_t *idle_rq = cpu_rq(cpu), *rq = cpu_rq(task_cpu(idle));
2646     unsigned long flags;
2647
2648     local_irq_save(flags);
2649     double_rq_lock(idle_rq, rq);
2650
2651     idle_rq->curr = idle_rq->idle = idle;
2652     deactivate_task(idle, rq);
2653     idle->array = NULL;
2654     idle->prio = MAX_PRIO;
2655     idle->state = TASK_RUNNING;
2656     set_task_cpu(idle, cpu);
2657     double_rq_unlock(idle_rq, rq);
2658     set_tsk_need_resched(idle);
2659     local_irq_restore(flags);
2660
2661     /* Set the preempt count _outside_ the spinlocks! */
2662     #ifdef CONFIG_PREEMPT
2663         idle->thread_info->preempt_count = (idle->lock_depth >= 0);
2664     #else
2665         idle->thread_info->preempt_count = 0;
2666     #endif
2667 }

```

第 2645 行：存储当前所在 CPU 的请求队列以及给定任务 `idle` 所在 CPU 的请求队列。本例中使用 `current` 和 `smp_processor_id()`，这些请求队列完全相等。

第 2648~2649 行：保存 IRQ 标志并获得这两个请求队列锁。

第 2651 行：设置运行 `idle` 任务的 CPU 请求队列的当前任务。

第 2652~2656 行：这些语句将任务 `idle` 从请求队列中移出，并移入 CPU 请求队列 `cpu` 中。

第 2657~2659 行：释放之前锁定的请求队列的锁。然后，将任务 `idle` 标记为重新调度，并恢复先前保存的 IRQ。最后，如果配置了内核抢占的话，则设置抢占计数器。

8.5.32 调用 `rest_init()`

第 493 行：例程 `rest_init()` 相当简单。事实上，它只创建一个称为 `init` 的线程，同时删除初始化内核锁，并调用 `idle` 线程：

```

-----
init/main.c
388 static void noinline rest_init(void)
389 {
390     kernel_thread(init, NULL, CLONE_FS | CLONE_SIGHAND);
391     unlock_kernel();
392     cpu_idle();
393 }
-----

```

第 388 行：读者可能已经注意到了，调用的第一个例程是 `start_kernel()` 而不是 `__init`。

回想一下，在第 2 章，我们说过当一个函数前加了 `__init` 时，表示用来保存函数变量等内容的内存，在初始化过程接近尾声时，这些内存就会被清除或释放。这可以通过调用 `free_initmem()` 来实现，分析 `init()` 时我们见过这个函数。`rest_init()` 不是 `__init` 函数的原因在于，它在完成前调用了 `init` 线程（即调用 `cpu_idle`）。因为 `init` 线程执行 `free_initmem()` 调用，所以可能存在竞态条件，这就是 `free_initmem()` 要在 `rest_init()`（或根线程）完成前被调用的原因。

第 390 行：本行创建 `init` 线程，通常也叫 `init` 进程或进程 1。简单地说，此处该线程和调用进程共享所有的内核数据结构。内核线程调用 `init()` 函数，在 8.6 节将讨论该函数。

第 391 行：当仅有单处理器时，例程 `unlock_kernel()` 什么都不做。否则，该函数释放大内核锁 BKL。

第 392 行：调用 `cpu_idle()` 将根线程转换为空闲线程。该例程将处理器让给调度程序，当调度程序没有其他挂起的进程可投入运行时，就返回该例程。

至此，已经完成了 Linux 内核初始化的大部分工作。下面，简要探讨 `init()` 函数。

8.6 init 线程（或进程 1）

现在，来看看 `init` 线程。注意，为简单起见，我们略过了所有与 SMP 相关的例程：

```
-----
init/main.c
601 static int init(void * unused)
602 {
603     lock_kernel();
...
612     child_reaper = current;
...
627     populate_rootfs();

629     do_basic_setup();
...
635     if (sys_access((const char __user *) "/init", 0) == 0)
636         execute_command = "/init";
637     else
638         prepare_namespace();
...
645     free_initmem();
646     unlock_kernel();
647     system_state = SYSTEM_RUNNING;

649     if (sys_open((const char __user *) "/dev/console", O_RDWR, 0) < 0)
650         printk("Warning: unable to open an initial console.\n");
651
652     (void) sys_dup(0);
653     (void) sys_dup(0);
...
662     if (execute_command)
663         run_init_process(execute_command);
```

```

664
665  run_init_process("/sbin/init");
666  run_init_process("/etc/init");
667  run_init_process("/bin/init");
668  run_init_process("/bin/sh");
669
670  panic("No init found. Try passing init= option to kernel.");
671  }

```

第 612 行: `init` 线程收养其父进程已经死亡的任何线程。变量 `child_reaper` 是一个全局指针, 在 `init/main.c` 中定义, 指向一个 `task_struct`。该变量准备承担“养父”的职责, 它所指向的线程将会成为新的父进程。提到诸如 `reparent_to_init()` (详细内容请查阅 `kernel/exit.c`), `choose_new_parent()` (详细内容请查阅 `kernel/exit.c`) 和 `forget_original_parent()` (详细内容请查阅 `kernel/exit.c`) 等函数是因为它们使用 `child_reaper` 来重置调用线程的父进程。

第 629 行: 函数 `do_basic_setup()` 用于初始化驱动程序模型, `sysctl` 接口, 网络套接字接口以及所支持的工作队列。

```

-----
init/main.c
551  static void __init do_basic_setup(void)
552  {
553      driver_init();
554
555      #ifdef CONFIG_SYSCTL
556          sysctl_init();
557      #endif
558
559      ...
560      sock_init();
561
562      init_workqueues();
563      do_initcalls();
564  }

```

第 553 行: 函数 `driver_init()` (详细内容请查阅 `drivers/base/init.c`) 初始化与驱动程序相关的所有子系统。这是设备驱动程序初始化的第一部分。第二部分见第 563 行的函数调用 `do_initcalls()`。

第 555~557 行: `sysctl` 接口为内核参数的动态转换提供支持。这意味着可以在运行时修改 `sysctl` 支持的内核参数, 而不需要重新编译和引导内核。`sysctl_init()` (详细内容请查阅 `kernel/sysctl.c`) 初始化该接口。有关 `sysctl` 的详情可参阅手册 (`man sysctl`)。

第 560 行: 函数 `sock_init()` 是个伪函数, 如果内核不支持网络的话, 该函数仅有 `printk` 语句, 此时, 该函数就在 `net/nonet.c` 中定义。如果内核支持网络功能, 则 `sock_init()` 在 `net/socket.c` 中定义, 作用是初始化用于支持网络功能的存储器高速缓存, 并注册支持网络的文件系统。

第 562 行: `init_workqueues` 设置工作队列通告程序链。第 10 章将讨论工作队列。

第 563 行: 函数 `do_initcalls()` (详细内容请查阅 `init/main.c`) 构成了设备驱动程序初始化的第二部分。该函数接着调用函数指针数组中的元素, 这些函数指针对应着内置的设备初始化

函数^①。

第 635~638 行：如果存在早期的用户空间 `init`，则内核并不准备命名空间，它允许执行这个函数。否则，调用函数 `prepare_namespace()`。命名空间指的是文件系统层次结构的挂载点。

```
-----
init/do_mounts.c
383 void __init prepare_namespace(void)
384 {
385     int is_floppy;
386
387     mount_devfs();
388
389     ...
391     if (saved_root_name[0]) {
392         root_device_name = saved_root_name;
393         ROOT_DEV = name_to_dev_t(root_device_name);
394         if (strncmp(root_device_name, "/dev/", 5) == 0)
395             root_device_name += 5;
396     }
397
398     is_floppy = MAJOR(ROOT_DEV) == FLOPPY_MAJOR;
399
400     if (initrd_load())
401         goto out;
402
403     if (is_floppy && rd_doload && rd_load_disk(0))
404         ROOT_DEV = Root_RAM0;
405
406     mount_root();
407 out:
408     umount_devfs("/dev");
409     sys_mount(".", "/", NULL, MS_MOVE, NULL);
410     sys_chroot(".");
411     security_sb_post_mountroot();
412     mount_devfs_fs ();
413 }
-----
```

第 387 行：函数 `mount_devfs()` 创建与挂载相关的结构 `/dev`。我们需要挂载 `/dev`，因为要使用它来查阅根设备的名称。

第 391~396 行：该代码块设置全局变量 `ROOT_DEV`，用来表示通过内核引导时参数传递进来的根设备。

第 398 行：主设备号的简单比较，以说明该根设备是不是软盘。

第 400~401 行：如果 RAM 盘是内核的根文件系统，则调用 `initrd_load()` 来挂载 RAM 盘。如果是这样的话，该函数返回 1 并跳转到标记 `out` 处，它将撤销来自设备的根文件系统准备过程中所作的一切工作。

第 406 行：调用 `mount_root` 来完成挂载根文件系统的主要工作。接下来深入分析一下该

① 关于 Trevor Woerner 对 `initcall` 机制更多精辟的论述可参阅 <http://geek.vtnet.ca/doc/initcall/>。

函数：

```
-----
init/do_mounts.c
353 void __init mount_root(void)
354 {
355 #ifdef CONFIG_ROOT_NFS
356   if (MAJOR(ROOT_DEV) == UNNAMED_MAJOR) {
357     if (mount_nfs_root())
358       return;
359
360     printk(KERN_ERR "VFS: Unable to mount root fs via NFS, trying
floppy.\n");
361     ROOT_DEV = Root_FD0;
362   }
363 #endif
364 #ifdef CONFIG_BLK_DEV_FD
365   if (MAJOR(ROOT_DEV) == FLOPPY_MAJOR) {
...
367     if (rd_doload==2) {
368       if (rd_load_disk(1)) {
369         ROOT_DEV = Root_RAM1;
370         root_device_name = NULL;
371       }
372     } else
373       change_floppy("root floppy");
374   }
375 #endif
376   create_dev("/dev/root", ROOT_DEV, root_device_name);
377   mount_block_root("/dev/root", root_mountflags);
378 }
-----
```

第 355~358 行：如果内核已挂载了 NFS 文件系统，则应该执行 `mount_nfs_root()`。如果挂载 NFS 失败，那么内核应输出适当的信息，并设法挂载软盘作为根文件系统。

第 364~375 行：在该代码块中，内核设法挂载根软盘^①。

第 377 行：该函数执行挂载根设备时的大部分工作。现在，让我们回到 `init()`。

第 645 行：调用 `free_initmem()`，释放那些带 `__init` 前缀的例程用完的所有内存段。这标志着退出纯内核空间，并开始设置用户模式的数据。

第 649~650 行：打开初始控制台。

第 662~668 行：变量 `execute_command` 在 `init_setup()` 中设置并保存引导参数的值，若不希望调用默认的 `/sbin/init` 的话，该参数包含调用 `init` 程序的名字。若传递了 `init` 程序的名字，该程序将获得比普通 `/sbin/init` 高的优先级。要注意的是，由于最后要调用 `execve()`，所以 `run_init_process()`（详细内容请查阅 `init/main.c`）并不返回值。因此，最初成功运行的 `init` 函数就是唯一运行的函数。如果没有找到 `init` 程序，可以使用 `bash shell` 来启动。

① 要注意 `rd_doload`：如果没有加载 RAM 盘，那么该全局变量的值为 0；如果已加载 RAM 盘，则其值为 1；“双 `initrd/ramload` 设置”时，其值为 2。

第 670 行：如果设法执行各种 `init` 程序的所有办法都失败了，那么将执行该错误语句。

内核的初始化到此为止。从现在开始，`init` 进程本身涉及系统的初始化，并启动所有必需的进程以及用户登录所需的后台支持。

8.7 小结

本章描述系统加电和内核引导期间发生了什么事情。我们讨论了 BIOS 和 Open Firmware，以及它们是如何与内核引导加载程序交互的。接下来讨论了几个最常用的引导加载程序 LILO、GRUB 和 Yaboot，并简单概括了它们如何工作，以及如何调用第一个内核初始化例程。

同时，我们也分析了建立内核初始化过程的函数。在整个初始化过程中，分析了内核代码，也谈到了前几章介绍的概念。特别是通过以下高级操作跟踪了 Linux 的内核初始化：

- ☐ 启动和锁住内核；
- ☐ 为 Linux 的内存管理初始化页高速缓存和页面地址；
- ☐ 为多 CPU 做好准备；
- ☐ 显示内核标志；
- ☐ 初始化 Linux 调度程序；
- ☐ 分析传到 Linux 内核的参数；
- ☐ 初始化中断处理程序、定时器处理程序和信号处理程序；
- ☐ 挂载初始文件系统；
- ☐ 完成系统的初始化，并将控制权从 `init` 交回系统。

离开内核初始化过程时，我们必须提到，内核还是在起作用的，它首先启动许多高级 Linux 应用程序，例如 X11、sendmail 等。所有这些程序都依赖于我们刚探讨过的基本结构和设置。

8.8 习题

1. 大内核锁 (BLK) 和一般自旋锁之间有什么区别？
2. `init` 脚本允许向 Linux 内核加入什么特殊的安全特性？
3. 谁为内核的页面管理初始化数据结构？
4. 触发后台程序将脏页写回磁盘时，脏页所占的百分比是多少？触发专门的写回程序时，脏页所占的百分比是多少？
5. 为什么是 `rest_init()` 不是 `__init` 函数？

第9章

构建 Linux 内核

本章内容

- 工具链
- 内核源代码的构建

到此为止，我们已经认识了 Linux 内核中的子系统，也探究了系统的初始化函数。同样，了解内核映像的创建也是非常重要的。本章将讨论内核映像的编译和链接过程，并考察内核的构建过程。

9.1 工具链

工具链 (toolchain) 是创建 Linux 内核映像的一组程序的集合。链的概念源于一个工具的输出将作为另一工具的输入这一特点。工具链包含编译程序、汇编程序和链接程序。从技术上讲，还需一个文本编辑器。本节只讨论前三个工具。开发软件时工具链必不可少，当然，SDK (软件开发工具包) 也是必需的。

编译程序 (compiler) 是一个转换程序，可将高级源语言转换成低级目标语言 (object language)。目标代码是运行在目标系统上并依赖于机器的一系列指令。**汇编程序** (assembler) 也是一个转换程序，可将汇编语言转换成与编译程序相同的目标代码。其不同之处在于：汇编程序产生的机器指令与汇编语句一一对应，而高级语言的每条语句可能对应多条机器指令。如你所知，Linux 源代码中与体系结构相关部分的某些文件是用汇编语言编写的，它们调用汇编程序将其编译为目标代码。

链接编辑程序 (link editor) 或**链接程序** (linker) 可将多个可执行模块链接成一个单元，以便执行。

图 9-1 说明了工具链的“链式关系”。链接程序将把程序的目标代码与它使用的所有函数库链接起来。编译程序中含有标志位，可供用户选择编译级别。例如，从图 9-1 中可以看到，编译程序可以直接产生机器代码，也可以先编译成汇编源代码，再经由汇编程序把它组合成计算机可直接执行的机器指令。

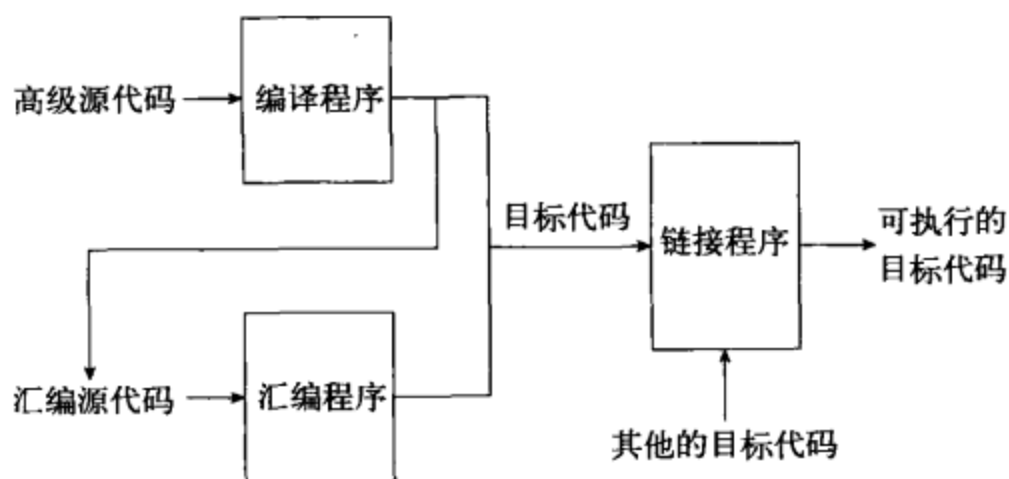


图 9-1 工具链

9.1.1 编译程序

常见的编译程序也具有链式特性，它分成多步执行，一步的输出作为下一步的输入，图 9-2 是其图解。编译的第一步是词法扫描 (scanner)，它将高级语言编写的程序拆分为一个个的单词符号 (tokens)。接下来，语法分析 (parser) 阶段根据一定的语法规则将这些符号组织起来，并由上下文分析阶段根据语义属性进一步组织这些符号。优化程序可提高所解析的符号的效率，代码生成阶段则产生目标代码。编译程序输出符号表和可重定位的目标代码。换句话说，每一个已编译的模块其起始地址都是 0，链接时必须重定位到合适的位置。

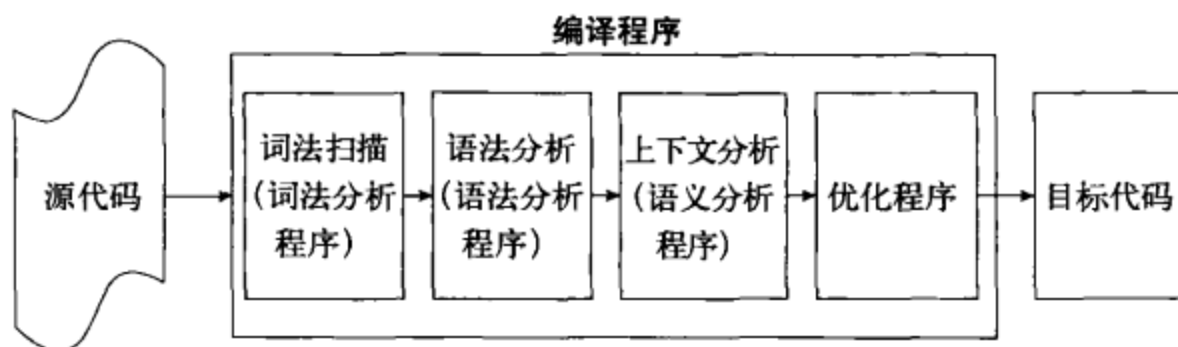


图 9-2 编译程序的操作过程

9.1.2 交叉编译

工具包通常自然地运行，这就意味着编译代码的系统和运行工具包生成的目标代码的系统相同。如果在 x86 系统上开发了一个内核，把它加载到另一个（或同样的）x86 系统上时，无须进行任何编译工作。功能强大的 Mac 和数量繁多的 x86 工具包可将代码编译成能在相应体系结构上运行的代码。但是，如果我们想在一种平台下编写代码，而在另一种平台下运行呢？

听来容易做起来难！考虑一下嵌入式市场，嵌入式系统通常是在有限内存和 I/O 接口的环境中实现其功能。无论控制的是汽车、路由器还是手机，这些设备都不足以容纳一个完整的嵌入式系统开发环境（更不用说监视器和键盘了）。解决的办法就是，让开发者利用他们功能强大、价格相对廉价的工作站作为宿主机系统 (host system) 来开发代码，然后将这些代码下载到目标系统 (target system) 上进行检测。因而称之为交叉编译。

例如，你可能是 PowerPC 嵌入式系统的开发者，该系统使用 405 处理器。而你的台式机的开

发系统绝大多数都是 x86 的。使用 gcc，你可以在这些台式机上完成所有开发工作（包括 C 程序和汇编程序），只要在编译时使用 `-mcpu=405` 选项^①就可以了。这样产生的目标代码具有 405 处理器特有的指令和寻址方式。之后就可以把可执行的目标代码下载到嵌入式系统中运行并调试。这样做看起来确实很乏味，但是对资源有限的嵌入式系统而言，它大大节省了内存。

在这种特殊环境下，许多针对交叉编译的嵌入式代码的辅助开发调试工具也应运而生。

9.1.3 链接程序

编译一个 C 程序时（例如，“Hello world!”），生成的代码远不止.c 文件中的这三四行。链接程序的工作就是找出所有引用的外部模块并链接起来。这些外部模块或函数库一般来自于开发者、操作系统和 C 运行库（`printf()` 的出处）。链接程序取出这些函数库，修订指针位置（**重定位**），并交叉引用模块中的**符号解析**，最终产生一个可执行模块。符号可以是全局的也可以是局部的。全局符号可以在模块内部定义，或由另一模块外部引用。链接程序要找到与模块相关的所有符号的定义。（注意，内核开发者无法使用用户空间库。）对于普通函数，内核自身有对应的函数。**静态库**是在链接时被找到并复制的，而**动态库**和**共享库**是在运行时才装载的，并让所有的进程共享。Microsoft 和 OS/2 将共享库称为动态链接库。Linux 提供的系统调用 `dlopen()`、`dlsym()` 及 `dlclose()` 用于加载/打开共享库，查找库中的符号，然后关闭共享库。

9.1.4 ELF 二进制目标文件

目标文件的格式随着生产商的不同而不同。现今，大多数 UNIX 系统都使用可执行链接格式 (ELF)。ELF 文件种类繁多，每种文件都执行不同的功能。主要的 ELF 文件类型有可执行文件、可重定位目标文件、核心文件及共享库。ELF 格式支持目标文件对不同平台和体系结构兼容。图 9-3 阐明可执行 ELF 目标文件和非可执行 ELF 目标文件。

非可执行 ELF 目标文件	可执行 ELF 目标文件
ELF 头	ELF 头
程序头表（可选）	程序头表（用于加载的节）
第 1 节	第 1 节
第 2 节	第 2 节
第 3 节	第 3 节
...	...
节头表 （用于链接的节）	节头表（可选）

图 9-3 可执行 ELF 文件和非可执行 ELF 文件

^① 要了解更多应用于 IBM RS/6000(POWER)及 PowerPC 的 gcc 选项，可登录 http://gcc.gnu.org/onlinedocs/gcc/RS_002f6000-and-PowerPC-Options.html#RS_002f6000-and-PowerPC-Options。

ELF 文件头总是位于 ELF 文件中偏移为 0 的位置，ELF 文件的所有信息均可通过 ELF 文件头获得。ELF 文件头是目标文件中唯一固定的结构，因而它必须能够定位并说明文件中其他子结构的位置和大小。所有 ELF 文件都被分为许多具有相似数据块的节 (section) 或段 (segment)。非可执行目标文件包含多个节和一个节头表 (section header table)，可执行目标文件则包含多个段和一个程序头表 (program header table)。

1. ELF 文件头

在 linux 系统中，数据结构 `elf32_hdr` 存储 ELF 文件头 (32 位的系统使用该结构，64 位的系统则使用 `elf64_hdr` 结构)。我们来看看这个结构：

```
-----
include/linux/elf.h
234 #define EI_NIDENT 16
235
236 typedef struct elf32_hdr{
237     unsigned char  e_ident[EI_NIDENT];
238     Elf32_Half     e_type;
239     Elf32_Half     e_machine;
240     Elf32_Word     e_version;
241     Elf32_Addr     e_entry; /* Entry point */
242     Elf32_Off      e_phoff;
243     Elf32_Off      e_shoff;
244     Elf32_Word     e_flags;
245     Elf32_Half     e_ehsize;
246     Elf32_Half     e_phentsize;
247     Elf32_Half     e_phnum;
248     Elf32_Half     e_shentsize;
249     Elf32_Half     e_shnum;
250     Elf32_Half     e_shstrndx;
251 } Elf32_Ehdr;
-----
```

第 237 行：e_ident 字段存放了一个 16 字节的魔数，用于标识该文件是否为 ELF 文件。

第 238 行：e_type 字段指定目标文件类型，例如，可执行文件、重定位文件、共享的目标文件等。

第 239 行：e_machine 字段表示被编译文件所在系统的体系结构。

第 240 行：e_version 字段表示目标文件的版本。

第 241 行：e_entry 字段保存程序的起始地址。

第 242 行：e_phoff 字段保存程序头表在文件中的偏移量（按字节计数）。

第 243 行：e_shoff 字段保存节头表在文件中的偏移量（按字节计数）。

第 244 行：e_flags 字段保存特定于处理器的标志。

第 245 行：e_ehsize 字段保存 ELF 头的大小。

第 246 行：e_phentsize 字段保存程序头表中每一项的大小。

第 247 行：e_phnum 字段包含程序头中表项的个数。

第 248 行：e_shentsize 字段保存节头表中每一项的大小。

第 249 行：e_shnum 字段保存节头中项的数量，表明该文件有多少节。

第 250 行：e_shstrndx 字段保存节头中节字符串的索引。

2. 节头表

节头表是 `Elf32_Shdr` 类型的数组，它在 ELF 文件中的偏移由 ELF 头的 `e_shoff` 字段来确定。在 ELF 文件中，每节都有一个对应的节头表：

```
-----
include/linux/elf.h
332  typedef struct {
333      Elf32_Word  sh_name;
334      Elf32_Word  sh_type;
335      Elf32_Word  sh_flags;
336      Elf32_Addr  sh_addr;
337      Elf32_Off   sh_offset;
338      Elf32_Word  sh_size;
339      Elf32_Word  sh_link;
340      Elf32_Word  sh_info;
341      Elf32_Word  sh_addralign;
342      Elf32_Word  sh_entsize;
343  } Elf32_Shdr;
-----
```

第 333 行：sh_name 字段包含节名。

第 334 行：sh_type 字段包含节的内容。

第 335 行：sh_flags 字段包含各种属性的信息。

第 336 行：sh_addr 字段保存节在内存映像中的地址。

第 337 行：sh_offset 字段保存 ELF 文件中这一节中初始字节的偏移量。

第 338 行：sh_size 字段包含节的大小。

第 339 行：sh_link 字段包含表链接 (table link) 的索引，该索引视其 sh_type 值而定。

第 340 行：sh_info 字段包含附加信息，视其 sh_type 值而定。

第 341 行：sh_addralign 字段包含地址对齐的约束。

第 342 行：当节中保存一张固定大小的表时，sh_entsize 字段包含节中每项的大小。

3. 非可执行 ELF 文件的节

ELF 文件被分成了多个节，每节都包含特定类型的信息。表 9-1 列出了这些节的信息类型，只有当在编译时设置了一定的编译器标志位，某些节才会出现。回忆一下，`ELF32_Ehdr->e_shnum` 保存了 ELF 文件中的节数。

表9-1 ELF文件节

节 名	说 明
.bss	未初始化的数据
.comment	GCC用于表示编译器版本的信息
.data	已初始化的数据
.debug	以符号表的形式给出的符号调试信息
.dynamic	动态链接信息
.dynstr	动态链接时的字符串
.fini	进程的终止代码，GCC的退出代码

(续)

节 名	说 明
.got	全局偏移量表
.hash	符号散列表
.init	初始化代码
.interp	程序解释器的路径名
.line	标记调试时的行号
.note	编译程序控制版本时所需的信息
.plt	过程链接表
.relname	重定位信息
.rodata	只读数据
.shstrtab	节名
.symtab	符号表
.text	可执行的指令

4. 程序头表

可执行的或共享的目标文件其程序头表是一个结构体数组。每个结构体描述一个段或执行该程序所需的其他信息：

```

-----
include/linux/elf.h
276 typedef struct elf32_phdr{
277     Elf32_Word  p_type;
278     Elf32_Off   p_offset;
279     Elf32_Addr   p_vaddr;
280     Elf32_Addr   p_paddr;
281     Elf32_Word   p_filesz;
282     Elf32_Word   p_memsz;
283     Elf32_Word   p_flags;
284     Elf32_Word   p_align;
285 } Elf32_Phdr;
-----

```

第 277 行：p_type 字段描述该段的类型。

第 278 行：p_offset 字段保存该段的开始相对于文件开始的偏移量。

第 279 行：若使用了该段，则 p_vaddr 字段保存该段的虚拟地址。

第 280 行：若使用了该字段，则 p_paddr 字段保存该段的物理地址。

第 281 行：p_filesz 字段保存文件映像中该段的字节数。

第 282 行：p_memsz 字段保存内存映像中该段的字节数。

第 283 行：p_flags 字段保存的标志依 p_type 而定。

第 284 行：p_align 字段描述要对齐的段在内存中如何对齐。该值是 2 的整数次幂。

通过这些信息，系统函数 exec() 和链接程序合作，为可执行程序在内存中创建进程映像，该过程包含如下步骤：

- 将可执行文件的段加入内存；
- 加载所有需要的共享库；
- 需要时重定位可执行文件及其共享对象；
- 将控制权交给程序；

理解目标文件的格式及可用的开发工具，有利于更好地调试编译时出现的问题（例如，未知的引用）。同时，了解代码载入和重定位之后的位置，有利于解决运行时出现的问题。

9.2 内核源代码的构建

现在，我们来讨论如何把内核编译成二进制映像文件，以及如何在执行前载入内存。作为一个内核开发者，你将会与源代码结下不解之缘，因而，有必要掌握如何阅读分析源代码以及如何编辑构建系统，以便添加自己的代码。

本节将带领你从下载源代码开始，直至教会你如何编译刚加载的内核映像文件。其中会谈到内核映像的创建，但不会是详细的、一步一步的指令手册式说明，网上许多关于如何构建内核映像的文档都很全面，例如内核 HOWTO (www.tldp.org/HOWTO/Kernel-HOWTO/)，该文档目前还在不断更新。它的目的是为将修改合并到构建系统，提供你所需要的信息。

程序开发者对构建系统或 Makefiles 并没有极大兴趣，但我们同样需要理解内核构建系统，以及如何通过修改源代码来更新内核。Linux 2.6 版的内核中又增添了许多工具帮助你更好地理解与内核构建系统有关的所有选项。同样，构建系统也经过较大地调整并重新设计，使之更有效地用文档记录。

本节将介绍如何布局源代码、如何构建内核以及 Makefiles 如何工作。第一步是获取源代码。我们先来看看源代码的结构，以及从何处获取源代码。

9.2.1 解释源代码

Linux 的官方源代码发布网址是 www.kernel.org。可下载的源代码有 gzip 压缩的.tar.gz 包和 bzip2 压缩的.tar.bz2 包。这些压缩包的源代码适用于任何体系结构。

内核开发者修改内核源代码后，必须将改动后的源代码提交给内核维护人员，由内核维护人员决定是否将这些改动加入下一个稳定树，最新的 PPC 开发通常在 www.penguinppc.org 的一棵独立的树下维护，所有改动都被保存在 PPC 树之下，当认为这些改动稳定而有价值时，就会被收录到主树中。目前，Linux 的 PPC 社团正在朝着主树进军。

源代码的位置在一定程度上取决于其发布版。例如，在红帽系统中，源代码被放置在 `/usr/src/linux-<version>/` 目录下（要么默认安装，要么被 RPM 安装）。若进行交叉编译，就是说，为某一体系结构构建的内核与进行实际编译的体系结构不同，源代码可能位于宿主机的某个 `/opt/<distribution name>/` 目录下，或在用户 chroot 到的根文件系统映像下。例如，面向嵌入式 Linux 市场的 Montavista 发布版，其源代码（和交叉编译程序）默认存储在 `/opt/mvista/` 目录下。

本节将源代码文件系统的根目录仅叫做根，在红帽发布版中，源代码的根目录位于 `/usr/src/linux-<version>/` 下。图 9-4 详细解释了源代码的结构化布局。

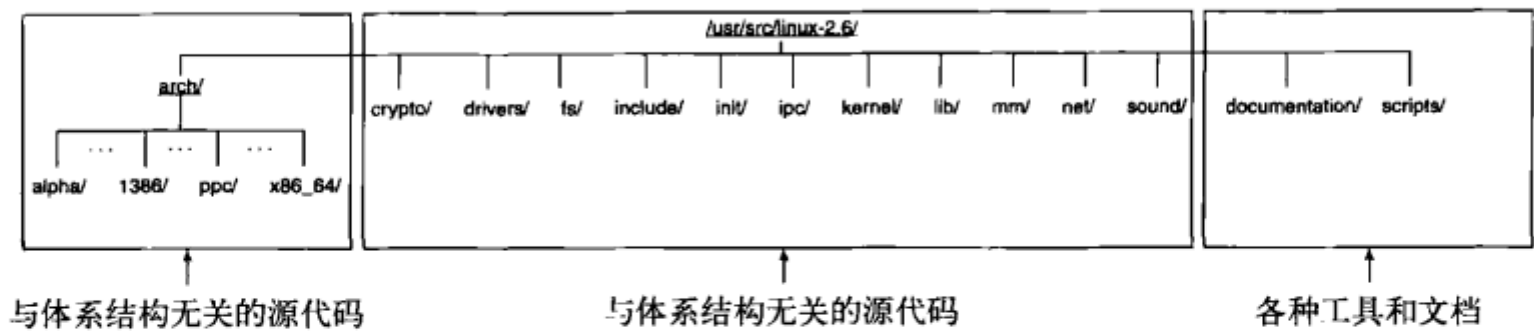


图 9-4 源代码的布局

源代码分为与体系结构相关的部分和与体系结构无关的部分。根目录下的 arch/ 目录包含了所有与体系结构相关的代码。该子目录下列出了从 kernel.org 镜像下载的源代码支持的所有体系结构。每一种支持的体系结构在 arch/ 下有一个相应的目录，其中包含与该体系结构相关代码的进一步细分。图 9-5 通过显示 arch/ 目录下的列表来展示它所支持的体系结构。

```
[1kp@ulaanbataar linux-2.6.7]$ ls arch/
alpha  arm26  h8300  ia64   m68knommu  parisc  ppc64  sh      sparc64  v850
arm     cris   i386   m68k   mips        ppc     s390   sparc   um       x86_64
[1kp@ulaanbataar linux-2.6.7]$
```

图 9-5 ls/usr/src/linux/arch

我们首先考察与结构体系无关的源代码，以便于对源代码的组织情况有个大致的了解，然后概览与体系结构相关的源代码，最后简要介绍不属于任何一类的其他各式各样的文件。

1. 与体系结构无关的代码

与体系结构无关的源代码根据其功能划分为 11 个不同的子目录。表 9-2 列出了这些子目录。

表9-2 与体系结构无关的子目录

子 目 录	目录说明
crypto	保存加密的API及各种加密/解密算法的代码
drivers	设备驱动程序的代码
fs	VFS及Linux支持的所有文件系统的代码
include	头文件代码。该目录下存放的子目录大都以前缀asm开头，用于存储特定于体系结构的头文件，其余子目录存放与体系结构无关的头文件
init	引导程序代码以及初始化代码中与体系结构无关的部分
ipc	支持进程间通信（IPC）的代码
kernel	内核空间特有的代码
lib	辅助函数的代码
mm	用于内存管理的代码
net	支持各种网络协议的代码
sound	支持音频系统的代码

前面多个章节中已经探究了其中的一个或多个子目录中的源代码。从上下文的角度考虑，以下各节仅简单回顾某些子目录，而略过那些未详细讨论过的子目录。

- fs/

fs/目录进一步分为支持VFS的C源文件和支援的各种文件系统的子目录。正如第7章中介绍的那样，VFS是对各种不同类型文件系统的抽象层，每个子目录下的代码将在存储设备与VFS抽象层之间架起一座桥梁。

- init/

init/目录中包含系统初始化时所有必需的代码。执行这些代码时，所有内核子系统都会被初始化，并创建所有初始进程。

- kernel/

大多数与结构体系无关的内核代码以及大部分内核子系统的代码都保存在kernel/目录下。某些代码有属于自己的目录，这些目录和kernel/目录处于同一层，例如文件系统和内存，根据文件名就可以知道它所包含的代码。

- mm/

mm/目录下保存用于内存管理的代码，在第4章已经见过这些代码的例子。

2. 与体系结构相关的代码

与体系结构相关的代码是指与硬件直接打交道的内核源代码。在你苦苦钻研这部分代码时，我们要提醒你的是Linux最初是为x86开发的。为了降低移植的难度，变量名和全局内核结构照搬了大多数x86核心术语。如果你浏览PPC代码时，遇到了PPC中不存在的与地址转换模式有关的名字时，不用大惊小怪。

当我们列出arch/i386/及arch/ppc目录下的文件时，你会发现有三个相同的文件：defconfig、Kconfig和Makefile。这三个文件都是与内核构建系统的基础结构有关的。9.2.2节将介绍这三个文件的作用。

表9-3列出了arch/ppc目录下的文件和子目录并进行了简单的介绍，如果你已经了解了Makefiles和Kconfig文件的结构，将有利于你浏览每个子目录中的这些文件，并有利于你熟悉代码的位置。

表9-3 arch/ppc/目录下的源码列表

子目录	目录说明
4xx_io	MPC4xx特有的I/O部分的源代码，特别是IBM STB3xxx SICC串口
8260_io	用于MPC8260-communication选项的源代码
8xx_io	用于MPC8xx-communication选项的源代码
amiga	用于PowerPC-equipped Amiga计算机的源代码
boot	与PPC引导程序相关的源代码。该目录下还包含一个images子目录，用于存储已编译的可引导映像
config	用于构建特定PPC平台和体系结构的配置文件
kernel	与内核子系统硬件相关的源代码
lib	PPC特殊库文件的源代码

(续)

子 目 录	目录说明
math-emu	PPC数学仿真器的源代码
mm	内存管理中特定于PPC部分的源代码，可参阅第6章的详细讨论
platforms	特定于安装PPC芯片的平台的源代码
syslib	特定于通用硬件的子系统的部分源代码
xmon	特定于PPC调试器的源代码

arch/x86 下的目录结构和与 PPC 体系结构相关的目录结构类似，表 9-4 概括了它所包含的各种子目录。

表9-4 arch/x86源码列表

子 目 录	目录说明
boot	与x86引导及安装过程相关的源代码
kernel	与内核子系统硬件相关的源代码
lib	x86库文件的源代码
mach-x	x86子结构的源代码
math-emu	x86数学仿真函数的源代码
mm	x86中内存管理部分的源代码，可参阅第6章的详细讨论
oprofile	oprofile内核分析工具的源代码
pci	x86 PCI驱动程序
power	x86电源管理的源代码

你也许会奇怪为什么两种体系结构的源码列表大相径庭，原因在于功能划分的差异，在一种体系结构上的划分不一定适合另一体系结构。例如，在 PPC 中，PCI 驱动程序会随着平台和子结构的不同而不同，要想在 PPC 平台下建立一个简单 PCI 的子目录，就要比在 x86 平台下困难得多。

3. 文件和目录杂项

在源根目录中的一些文件，既和与体系结构相关的代码无关，又和与体系结构无关的代码无关，表 9-5 列出了这些文件。

表9-5 其他文件

文件/目录	说 明
COPYING	Linux的GPL许可证
CREDITS	列出了对Linux做出贡献的人名
MAINTAINERS	当提交内核改动时涉及的维护人员及说明
README	发布说明
REPORTING-BUGS	描述报告bug的过程
documentation/ scripts/	包含了Linux内核及其源代码的有关文档，当内核有所更新时，这里含有丰富的改动信息 构建内核的过程中可以使用的功能及脚本

9.2.2 构建内核映像

内核构建系统或者说 kbuild, 是一种在构建内核时, 可以对内核配置选项进行选择的机制。2.6 内核树中已经更新了这种机制, 新版本的 kbuild 不仅高速而且备有更完善的文档。kbuild 系统完全依赖于源代码的层次结构。

1. 内核配置工具

利用内核配置工具自动生成名为 .config 的内核配置文件, 这是构建内核的第一步。 .config 文件位于源代码根目录下, 该文件中包含所有内核配置选项的描述, 可以借助内核配置工具来选择这些选项。每个内核配置选项都有相关的名字和值。其名字的形式为 CONFIG_<NAME>, 其中 <NAME> 是对相关选项的标识; 变量可以有三个值: y、m 或 n。y 代表 “yes”, 表示该选项将会被编译到内核源代码中, 或者说会被构建到系统中。m 代表 “module”, 表示该选项将会以独立于内核源代码的模块的方式编译到内核中。如果未选择该选项 (即将该选项的变量值设为 n, 代表 “no”), 那么 .config 文件中就会出现下列注释: CONFIG_<NAME> is not set。 .config 文件中的选项的位置根据它们在内核配置工具中的顺序进行排序, 注释部分说明该选项位于哪个菜单下。我们来看看一个 .config 文件的节选:

```
-----
.config
1  #
2  # Automatically generated make config: don't edit
3  #
4  CONFIG_X86=y
5  CONFIG_MMU=y
6  CONFIG_UID16=y
7  CONFIG_GENERIC_ISA_DMA=y
8
9  #
10 # Code maturity level options
11 #
12 CONFIG_EXPERIMENTAL=y
13 CONFIG_CLEAN_COMPILE=
14 CONFIG_STANDALONE=y
15 CONFIG_BROKEN_ON_SMP=y
16
17 #
18 # General setup
19 #
20 CONFIG_SWAP=y
21 CONFIG_SYSVIPC=y
22 #CONFIG_POSIX_MQUEUE is not set
23 CONFIG_BSD_PROCESS_ACCT=y
-----
```

上述 .config 文件指出第 4~7 行的选项位于顶层菜单中, 第 12~15 行的选项位于代码成熟度选项菜单中, 第 20~23 行的选项位于通用设置选项菜单中。

所有配置工具都会产生上述菜单, 并且已经看到代码成熟度菜单项及通用设置菜单项等选项都位于顶层。后面两个选项被扩展为包含多个选项的子菜单。这些菜单都是在调用 make xconfig

命令时，由 qconf 配置工具提供的。配置工具显示的菜单都默认用于 X86 体系结构。图 9-6 给出了 PPC 体系结构所使用的菜单，在 make xconfig 调用的末尾，须追加参数 ARCH=ppc。

调用 make bzImage 构建内核映像时，顶层 makefile 文件会读取由配置工具产生的.config 文件。顶层 makefile 文件中包含了特定体系结构的 Makefile 文件提供的信息，它位于 arch/<arch>/目录下，该 Makefile 文件中包含了与平台相关的信息，通过 include 指令来实现。

```
-----
Makefile
434 include .config
...
450 include $(srctree)/arch/$(ARCH)/Makefile
-----
```

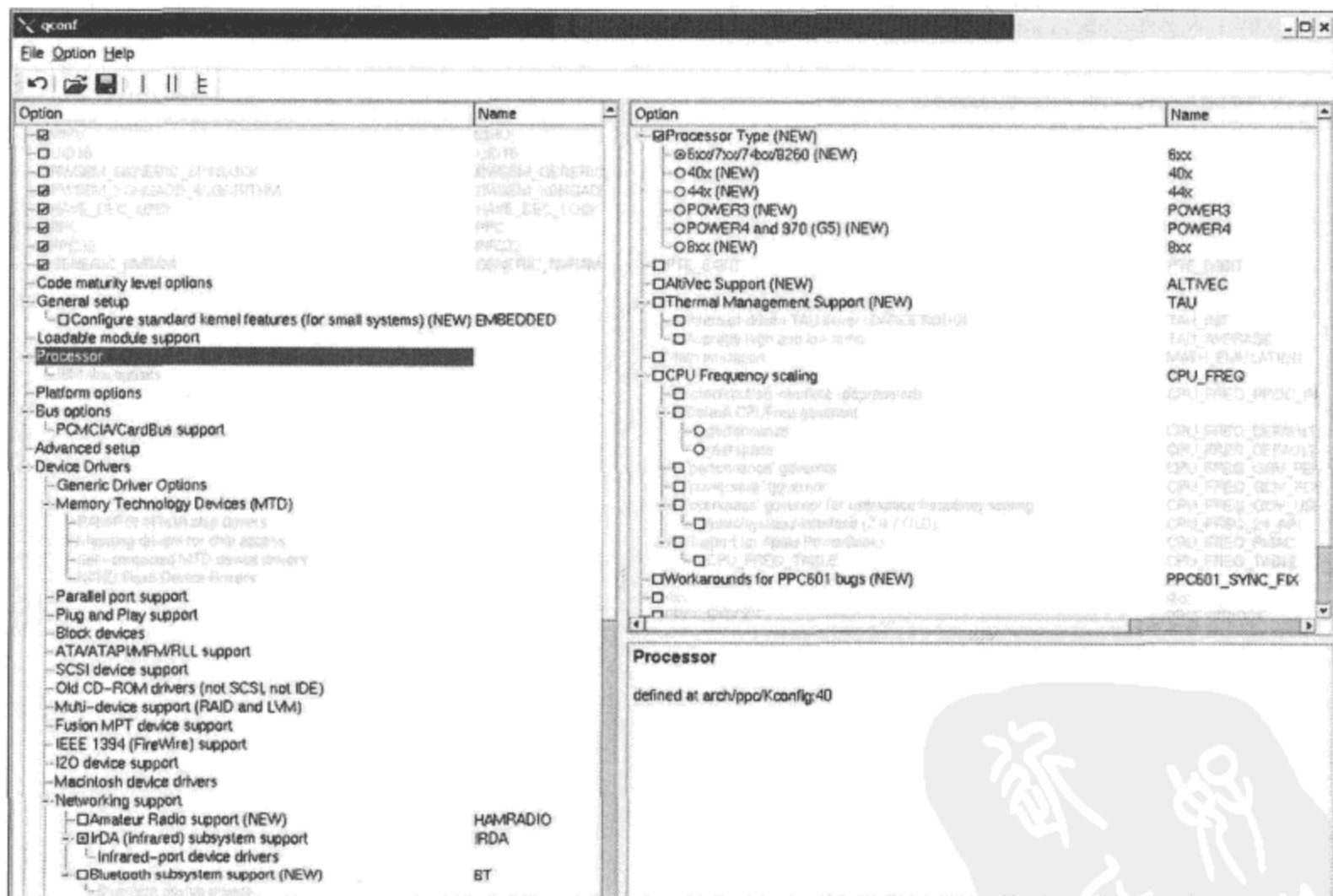


图 9-6 qconf 快照

由此可见，Makefile 文件已经确定编译的目标体系结构了。顶层 Makefile 文件通过以下三种可能的方式来确定体系结构：

- (1) 借助命令行参数 ARCH；
- (2) 借助环境变量 ARCH；
- (3) 在执行构建的宿主机上调用 uname，并自动接收由此命令产生的信息。

如果编译时所面向的体系结构与执行编译过程的宿主机体系结构不同时，就必须传递参数 CROSS_COMPILE，这是一个前缀，表示使用交叉编译器，或者，通过自行编辑 Makefile，并给该

变量赋值。例如，当在 x86 宿主机上为 PPC 处理器编译代码时，将执行下面的命令：

```
lkp:~#make xconfig ARCH=ppc
lkp:~#make ARCH=ppc CROSS_COMPILE=ppc-linux-
```

.config 文件也会产生 include/linux/autoconf.h 文件，#define 宏中包含已选择的 CONFIG_<NAME>值，而#undef 宏包含未选择的 CONFIG_<NAME>值。

2. 子makefile文件

构建一个系统依赖于各个子目录下的子 Makefile 文件。每个子目录下的 Makefile(被称为 sub-Makefile 或 kbuild Makefile) 都定义了根据该子目录下的源码文件编译目标文件的规则，并且仅对该目录下的文件作适当的修改。我们采用递归的方式调用位于 init/、drivers/、sound/、net/、lib/ 和 usr/等目录下的各个子目录中的子 Makefile 文件。

在递归调用之前，kbuild 首先要确定是否已经满足一些必要的条件，包括在必要时更新 include/linux/version.h 文件，并设置符号链接 include/asm，使之指向我们所编译的目标体系结构特有的文件。例如，如果为 PPC 编译代码，则 include/asm 指向 include/asm-ppc。kbuild 还要对文件 include/linux/autoconf.h 和 include/linux/config 进行编译。之后，从根目录开始进行递归。

如果你是一个内核开发者，当你向某一特殊的子系统添加了自己的代码时，要将你的文件放在一个特定的子目录中，并在必要时更新 makefile 文件以包括一些修改。如果你的代码是添加在一个已有的文件中，就可以使用 #ifdef(CONFIG_<NAME>) 块来把你的代码包围起来。如果在 .config 文件中选中这个值，那么在 include/linux/autoconf.h 文件中就包含相应的 #define 语句，就说明你的宏是有效的，而且在编译时会把你的修改包含进去。

我们必须遵守子 Makefile 文件特有的格式，它指出该如何编译目标文件。这些子 Makefile 文件都很简单，因为诸如编译程序名、要使用的函数库等都已经顶层 Makefile 文件以及与特定于体系结构的顶层 Makefile 文件中定义了，要遵循的编译规则在 scripts/Makefile.*s 文件中定义。子 Makefile 文件可能编译的文件列表如下。

□ \$(obj-y)：将被链接到 built-in.o，最终编译到 vmlinux 的目标文件列表。

□ \$(obj-m)：编译成模块的目标文件列表。

□ \$(lib-y)：编译到 lib.a 的目标文件列表。

换句话说，当我们执行 make bzImage 类型的调用时，kbuild 构建所有属于 obj-y 类型的目标文件并将它们链接到一起。所以子 Makefile 文件的要素之一就是指明类型。

```
obj-$(CONFIG_FOO) += foo.o
```

如果顶层 Makefile 文件读入的 .config 文件中的 CONFIG_FOO 被设置成 y，那么这一行就等价于 obj-y += foo.o。随后，kbuild 根据 scripts/Makefile.build 文件中定义的规则将该目录下的 foo.c 或 foo.S 文件编译为目标文件（稍后还有 scripts/Makefile.build 文件的更多介绍）。若 foo.c 或 foo.S 文件不存在，则显示如下语句：

```
Make[1]: *** No rule to make target '<subdir>/foo.o', needed by
'<subdir>/built-in.o'. Stop.
```


kbuild 通过对 obj-y 或 obj-m 显式地附加条件进行递归。你可以添加一个目录来设置 obj-y, 表明 kbuild 将会访问这个指定目录:

```
Obj-$(CONFIG_FOO) += /foo
```

若/foo 不存在, 则显示如下语句:

```
Make[2]: *** No rule to make target '<dir>/foo/Makefile'. Stop.
```

CML2

选择内核选项时, 你有没有想过我们所使用的配置工具在何处获得相关信息? kbuild 是以 CML2 为基础的。CML2 是一种专门用于内核配置的特定领域的语言, 它产生规则库 (rulebase), 解释器将读取并利用这些规则产生 config 文件, 该 config 文件包含 CML2 语言的语法规则和语义。配置程序读取的 CML2 规则库存放在 defconfig 和 Kconfig 文件中。defconfig 文件位于特定体系结构的顶层目录 arch/*/中, Kconfig 文件在其他大多数子目录中都可以找到, 该文件包含了创建的选项信息, 例如应该在该文件中列出的菜单、应提供的帮助信息、config 名称值, 以及只编译到内核中还是同时编译成模块的选择信息。有关 CML2 和 Kconfig 文件的更多信息, 详情可参阅文件 Documentation/kbuild/kconfig-language.txt。

让我们回顾一下有关 kbuild 操作的执行过程, 第一步是通过命令 make xconfig 或 make xconfig ARCH=ppc 调用配置工具进行配置, 使用哪个命令取决于构建过程所面向的体系结构。我们对该工具进行的选择会被保存在.config 文件中。发出一个诸如 make bzImage 的内核构建调用时, 顶层的 Makefile 文件会读取.config 的内容。顶层 Makefile 文件向下一级目录递归之前, 将执行以下操作:

- (1) 更新 include/linux/version.h.
- (2) 设置符号链接 include/asm, 使之指向编译过程所面向的特定目标体系结构的文件。
- (3) 构建 include/linux/autoconf.h.
- (4) 构建 include/linux/config.h.

随后 kbuild 操作会递归到下一级目录中, 对所有的子 Makefile 文件进行 make 操作, 并在每个子目录中创建目标文件。

我们已经讨论了子 Makefile 文件的结构, 现在我们来研究一下顶层 Makefile 文件, 看看驱动内核编译时如何使用它。

3. Linux内核的Makefile文件

Linux 的 Makefile 文件相当复杂。本节旨在说明源码树下所有子 Makefiles 文件之间的关系, 以及在这些文件中执行 make 命令的细节。但是, 如果你想通过理解 kbuild Makefile 文件的所有细节, 来拓展有关 make 知识, 是不错的起点。有关 make 的更多信息, 可以登录 www.gnu.org/software/make/make.html。

事实上, 源码树的几乎每个子目录下都有一个 makefile 文件。如前所述, 子树的 Makefile 主要负责特殊种类的源码 (或内核子系统), 这些文件相当简单, 并仅定义了要加入源码表的目标源文件, 稍后再编译这些文件。同时, 另有 5 个其他 Makefile 文件定义规则并执行这些文件, 它

们就是顶层 makefile 文件：arch/\$(ARCH)/Makefile、scripts/Makefile.build、scripts/Makefile.clean 和 scripts/Makefile。图 9-7 给出了各个 Makefile 文件间的关系。可定义的关系类型有“include”和“execute”。提及“include”关系类型时，是指 Makefile 要通过 include <filename> 规则添加其他文件的信息。提及“execute”关系类型时，是指上一级 Makefile 通过 make-f 命令调用它的下一级 makefile 文件。

在源码树的根目录下发出一个 make 调用时，就会调用顶层 Makefile 文件。顶层 Makefile 文件定义了随后要输出到其他 Makefile 的变量，还进一步对顶层源代码子目录中的每个 Makefile 发出 make 调用，终止对它们执行的操作。

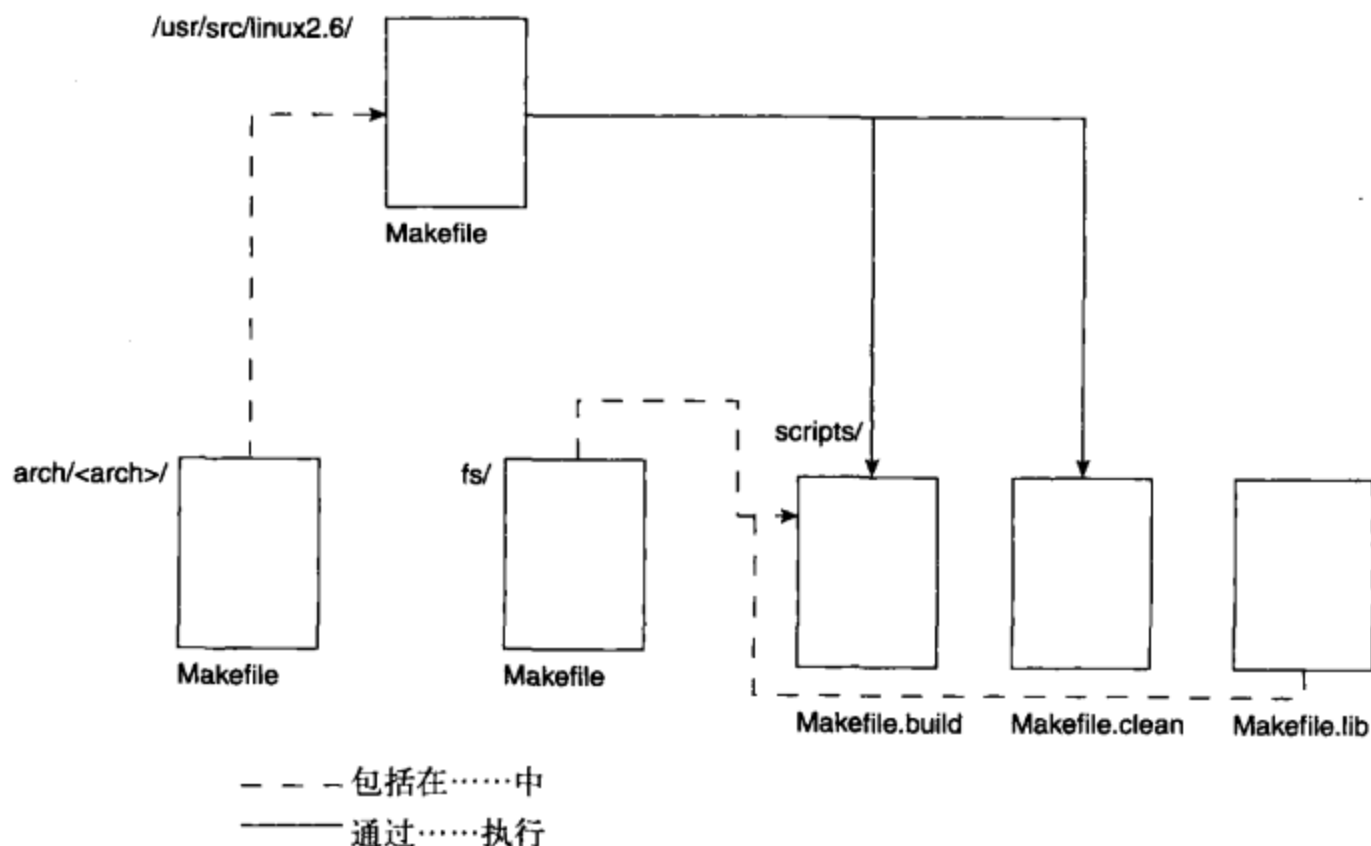


图 9-7

编译程序和链接程序的调用在 scripts/Makefile.build 中定义。就是说，向下一级子目录递归并通过 make 命令编译目标文件时，其实是执行了定义在文件 Makefile.build 中的一些规则。这可以借助 \$(Q) \$(MAKE) \$(build)=<dir> 的简写形式的调用来实现，这条规则就是 make 在每个子目录中得以调用的方式。build 变量被简写为：

```

Makefile
1157 build := -f $(if $(KBUILD_SRC),$(srctree)/)scripts/Makefile.build obj
-----
  
```

调用 \$(Q) \$(MAKE) \$(build)=fs 可扩展为：

```
"@ make -f /path/to/source/scripts/Makefile.build obj=fs".
```

随后，scripts/Makefile.build 读取作为参数（本例中是 fs）传递的目录中的 Makefile 文件。子 Makefile 文件中定义了一个或多个 obj-y、obj-m、lib-y 等列表。之后，文件 scripts/Makefile.build 会根据 scripts/Makefile.lib 文件中的定义，对这些子目录中的源文件进行编译，然

后再进一步递归到列表中定义的任意子目录，调用的方式与前面相同。

举例说明，若我们可以通过内核配置工具得到文件系统菜单，并选择了 Ext3 日志文件系统，那么.config 文件中的 CONFIG_EXT3_FS 就会被设置为 y，下面是子 Makefile 文件中相应 fs 的片断：

```
-----
Makefile
49  obj-$(CONFIG_EXT3_FS) += ext3/
-----
```

当 make 使用这种规则运行时，会进行 obj-y += ext3/ 操作，即把 ext3/ 作为 obj-y 中的一项。make 自动辨识出它是一个子目录，并调用 \$(Q) \$(MAKE) \$(build)=ext3。

\$(Q)

在所有 \$(MAKE) 调用前都要加一个 \$(Q) 变量。对于 2.6 内核树及 kbuild 基础结构的整理，你可以禁止 make 冗长的输出格式。make 在执行之前先输出命令行。当某一行以 @ 开头时，那么表示不必输出这一行：

```
-----
Makefile
254  ifeq ($(KBUILD_VERBOSE),1)
255    quiet =
256    Q =
257  else
258    quiet=quiet_
259    Q = @
260  endif
-----
```

从上述几行可以看出，如果 KBUILD_VERBOSE 被设置为 0 则 Q 被设置为 @，目的是简化编译语句。

当构建过程结束后，就会产生一个内核映像文件。因为内核是通过 zlib 算法进行压缩的，故可启动的压缩内核映像文件被称作 zImage 或 vmlinuz。常见的 Linux 规范也指定可引导映像 在文件系统中的位置，一般必须位于 /boot 或 / 目录下。到此，内核映像文件就可以通过引导加载 程序装载到内存中。

9.3 小结

本章探究目标文件的编译、链接过程以及目标文件的结构，以便理解可执行代码的最终形式。同时我们考察了围绕内核编译系统的一些基本结架，以及源代码的结构如何与构建系统本身相结合。由于前面章节已有相关讨论，本章只作简要介绍。

9.4 习题

1. 描述不同类型的 ELF 文件，并说明它们的作用。

2. 目标文件中的段是指什么?
3. 考察文件 `arch/ppc/Kconfig` 及 `arch/i386/Kconfig`, 给出每种体系结构所支持的处理器类型。
4. 考察文件 `arch/ppc` 及 `arch/i386`, 它们所共有的文件和目录有哪些? 研究这些文件并列出它们提供的支持, 看看它们是否完全匹配?
5. 如果你要交叉编译内核, 那么你给交叉编译程序指定前缀时所使用的参数是什么?
6. 在什么情况下, 你要通过命令行参数 `ARCH` 来指定系统的体系结构?
7. 什么是子 `makefile` 文件? 它们如何工作?
8. 考察文件 `scripts/Makefile.build`, `scripts/Makefile.clean` 以及 `scripts/Makefile.lib`, 分别列举其功能。



第 10 章

向内核添加代码

本章内容

- 浏览源代码
- 编写代码
- 构建和调试

本章分为“浏览源代码”和“编写源代码”两大部分。

10.1 节描述几乎所有 Linux 系统都通用的一个设备驱动程序，它存放于目录/dev/random 下，并说明内核如何与之通信。同时，再次像以前一样描述内核的内部工作，并展示其实用之处。

10.2 节将构建一个设备驱动程序，并深入分析编写设备驱动程序时会遇到的常见问题。

此后将讲述如何使用/proc 系统来调试设备驱动程序。也许这就是一个硬币的第三面？

10.1 浏览源代码

本节将介绍 Linux 下的系统调用及设备驱动程序（也称为模块）的概念。用户程序通过系统调用向操作系统请求服务，因而可以通过添加系统调用来创建新的内核服务。第 3 章描述了系统调用的内部实现。本章将介绍如何将自定义系统调用合并到 Linux 内核中，从而展示其实用的一面。

设备驱动程序涉及 Linux 内核利用的接口，让开发者可以控制系统的输入/输出设备。全书都是围绕 Linux 设备驱动程序来撰写的，而本章则是这一主题的精华所在。本节沿着这样一个思路来介绍驱动程序，首先说明在文件系统中如何表示设备，然后分析控制设备的具体内核代码。下一节将介绍如何利用所学知识构建一个实用的字符驱动程序。本章的最后将介绍如何编写系统调用，以及如何构建内核。我们首先来研究一下文件系统，然后说明文件如何与内核相关联。

10.1.1 熟悉文件系统

通过/dev 可以访问 Linux 的设备。例如，执行命令 `ls -l /dev/random`，生成的结果如下：

```
crw-rw-rw- 1 root  root  1, 8 Oct 2 08:08 /dev/random
```

首字母 c 表示该设备是一个字符设备，若为 b 则表示块设备。接下来的两列是用户及用户所在的组，后面是以逗号隔开的是两个数字（这里是 1 和 8），第一个数字是设备的主设备号，第

二个是从设备号。设备驱动程序向内核注册时就注册了其主设备号。打开一个给定的设备时，内核将通过设备文件的主设备号^①找到以主设备号注册的设备驱动程序。从设备号通过内核传递给设备驱动程序本身，这是因为一个单独的驱动程序可以控制多个设备。例如，`/dev/urandom` 设备的主设备号为 1 而从设备号为 9，表示以主设备号为 1 注册的设备驱动程序可以处理设备 `/dev/random` 和 `/dev/urandom`。

要产生一个随机数，只需读取 `/dev/random`。下面是读取 4 字节随机数^②的一种方法：

```
lkp@lkp:~$ head -c4 /dev/urandom | od -x
0000000 823a 3be5
0000004
```

若重复执行这条命令，会发现产生的 4 字节随机数[823a 3be5]总在变化。为了举例说明 Linux 内核如何使用设备驱动程序，我们来分析用户访问 `/dev/random` 时内核的操作步骤。

我们已经知道 `/dev/random` 设备文件的主设备号是 1，并可以通过查看 `/proc/devices` 文件来确定驱动程序控制哪个设备节点。

```
lkp@lkp:~$ less /proc/devices
Character devices:
1 mem
```

让我们以 `mem` 设备驱动程序为例来看看“随机数”是如何产生的。

```
-----
drivers/char/mem.c
653 static int memory_open(struct inode * inode, struct file * filp)
654 {
655     switch (imajor(inode)) {
656         case 1:
657             ...
676         case 8:
677             filp->f_op = &random_fops;
678             break;
679         case 9:
680             filp->f_op = &urandom_fops;
681             break;
-----
```

第 655~681 行：switch 条件语句根据操纵的设备的从设备号来初始化驱动程序的数据结构，尤其是设置 `filp` 和 `fops`。

我们不禁想问：“什么是 `flip`? 什么是 `fop`?”

10.1.2 `flip` 和 `fops`

`flip` 只是一个文件结构指针，而 `fop` 是一个 `file_operations` 结构指针。内核通过 `file_operations` 结构来确定操作文件时要调用的函数。下面是 `file_operations` 结构中用于随机设备驱动程序的部分内容。

① 使用命令 `mknod` 可创建块设备文件和字符设备文件。

② `head-c4` 用于聚集前 4 字节，`od-x` 以十六进制来表示获得的随机数。


```

-----
include/linux/fs.h
556 struct file {
557     struct list_head f_list;
558     struct dentry *f_dentry;
559     struct vfsmount *f_vfsmnt;
560     struct file_operations *f_op;
561     atomic_t f_count;
562     unsigned int f_flags;
...
581     struct address_space *f_mapping;
582 };
-----

include/linux/fs.h
863 struct file_operations {
864     struct module *owner;
865     loff_t (*llseek) (struct file *, loff_t, int);
866     ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
867     ssize_t (*aio_read) (struct kiocb *, char __user *, size_t, loff_t);
868     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
869     ssize_t (*aio_write) (struct kiocb *, const char __user *, size_t, loff_t);
870     int (*readdir) (struct file *, void *, filldir_t);
871     unsigned int (*poll) (struct file *, struct poll_table_struct *);
872     int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
...
888 };
-----

```

随机设备驱动程序以如下方式声明它所提供的文件操作：驱动程序所实现的函数必须符合 `file_operations` 结构中所列出的函数原型：

```

-----
drivers/char/random.c
1824 struct file_operations random_fops = {
1825     .read    = random_read,
1826     .write   = random_write,
1827     .poll    = random_poll,
1828     .ioctl   = random_ioctl,
1829 };
1830
1831 struct file_operations urandom_fops = {
1832     .read    = urandom_read,
1833     .write   = random_write,
1834     .ioctl   = random_ioctl,
1835 };
-----

```

第 1824~1829 行：随机设备提供的操作有 `read`、`write`、`poll` 以及 `ioctl`。

第 1831~1835 行：`urandom` 设备提供的操作有 `read`、`write` 以及 `ioctl`。

`poll` 操作允许程序员在执行某种操作之前查看该操作是否阻塞。它指出当请求的熵的字节

多于熵池^①中的字节时，`/dev/random` 就会阻塞，事实也的确如此。但 `/dev/urandom` 设备不会被阻塞，只是当熵池太小时，它可能无法返回完整的随机数。详情可查看系统的帮助页面，具体来说，是使用命令 `man 4 random`。

进一步研究代码时发现，对 `/dev/random` 进行读操作时，内核将控制权交给函数 `random_read()` (参见代码第 1825 行)。`random_read()` 定义如下：

```
-----
drivers/char/random.c
1588 static ssize_t
1589 random_read(struct file * file, char __user * buf, size_t
nbytes, loff_t * ppos)
-----
```

该函数的参数如下。

- **file**: 指向设备文件的结构体。
- **buf**: 指向存放结果的用户空间。
- **nbytes**: 请求数据的大小。
- **ppos**: 指向文件中用户访问的位置。

也许你会想到这样一个问题：如果设备驱动程序在内核空间中执行，但缓冲区却位于用户空间，那我们如何才能安全访问 `buf` 中的数据？下一节将解释数据在用户空间和内核空间之间移动的过程。

10.1.3 用户空间和内核空间

若想仅通过 `memcpy()` 便把数据从内核空间复制到用户空间，未必行得通，因为执行 `memcpy()` 操作时用户空间的地址可能已被交换出去。Linux 提供的函数 `copy_to_user()` 及 `copy_from_user()` 使得驱动程序可以在内核空间和用户空间中传递数据。在函数 `read_random()` 中，通过 `extract_entropy()` 函数来实现这一功能，但还有一些曲折：

```
-----
drivers/char/random.c
1: static ssize_t extract_entropy(struct entropy_store *r, void * buf,
2:      size_t nbytes, int flags)
3: {
1349 static ssize_t extract_entropy(struct entropy_store *r, void * buf,
1350      size_t nbytes, int flags)
1351 {
...
1452      /* Copy data to destination buffer */
1453      i = min(nbytes, HASH_BUFFER_SIZE*sizeof(__u32)/2);
1454      if (flags & EXTRACT_ENTROPY_USER) {
1455          i -= copy_to_user(buf, (__u8 const *)tmp, i);
1456          if (!i) {
1457              ret = -EFAULT;

```

① 在随机设备驱动程序中，熵指不可预测的系统数据，常见的熵的来源有：键盘输入，鼠标移动以及其他无规律的输入等。

```

1458         break;
1459     }
1460 } else
1461     memcpy(buf, (__u8 const *)tmp, i);
-----

```

extract_entropy() 包含如下参数：

- **r** 是指向熵内部存储器的指针，为了便于讨论我们不予考虑；
- **buf** 是指向内存区的指针，该内存区应当已填充数据；
- **nbytes** 是写入 **buf** 的数据大小；
- **flags** 告诉函数 **buf** 是在内核空间还是用户空间；
- **extract_entropy()** 将返回 `ssize_t`，表明已产生的随机数占多少字节。

第 1454~1455 行：若 **flags** 表明指针 **buf** 指向用户空间中的某个位置，那么我们将使用函数 `copy_to_user()` 将内核空间的内容复制给用户空间，其中指针 **tmp** 和 **buf** 分别指向内核空间和用户空间。

第 1460~1461 行：若指针 **buf** 指向内核空间的某一位置，则可使用函数 `memcpy()` 来复制数据。

获得随机字节是内核空间以及用户空间程序都可能涉及的事情；内核空间的程序可以通过不设置标志位来避免函数 `copy_to_user()` 带来的额外开销。例如，内核实现一个加密的文件系统，可以避免把数据复制到用户空间的额外开销。

10.1.4 等待队列

我们先不考虑数据如何在内核空间和用户空间之中传输，而是回到 `read_random()` 函数，看看它如何使用等待队列。

有时，驱动程序可能需要等待某些条件的成立，也许要访问系统资源。此时，我们并不希望内核一味等待访问的结束。当等待发生时，因为所有系统处理都得暂停而引起内核等待就会出现^①。声明一个等待队列，可以将等待某种条件的驱动程序延迟到条件满足时再进行处理。

正在等待的进程使用了两个数据结构，即等待队列和等待队列头。模块应当创建等待队列头，并利用使用宏 `sleep_on`、`wake_up` 的模块部分进行管理。函数 `random_read()` 恰是如此：

```

-----
drivers/char/random.c
1588 static ssize_t
1589 random_read(struct file * file, char __user * buf, size_t nbytes, loff_t
*ppos)
1590 {
1591     DECLARE_WAITQUEUE(wait, current);
...
1597     while (nbytes > 0) {
...
1608         n = extract_entropy(sec_random_state, buf, n,
1609                             EXTRACT_ENTROPY_USER |

```

^① 实际上，运行内核任务的 CPU 会暂停运行。对于多处理器系统来说，其他 CPU 尚可继续运行。

```

1610         EXTRACT_ENTROPY_LIMIT |
1611         EXTRACT_ENTROPY_SECONDARY);
...
1618     if (n == 0) {
1619         if (file->f_flags & O_NONBLOCK) {
1620             retval = -EAGAIN;
1621             break;
1622         }
1623         if (signal_pending(current)) {
1624             retval = -ERESTARTSYS;
1625             break;
1626         }
...
1632         set_current_state(TASK_INTERRUPTIBLE);
1633         add_wait_queue(&random_read_wait, &wait);
1634
1635         if (sec_random_state->entropy_count / 8 == 0)
1636             schedule();
1637
1638         set_current_state(TASK_RUNNING);
1639         remove_wait_queue(&random_read_wait, &wait);
...
1645         continue;
1646     }

```

第 1591 行：初始化当前任务的等待队列 `wait`。宏 `current` 是一个指针，指向当前任务的 `task_struct` 结构。

第 1608~1611 行：从设备获取一组随机数据。

第 1618~1626 行：若无法从熵池中获取所需数目的熵，并且未被阻塞或有挂起的信号时，给调用者返回错误信息。

第 1631~1633 行：建立等待队列。`random_read()` 使用自己的等待队列 `random_read_wait`，而不是系统的等待队列。

第 1635~1636 行：此时，读操作被阻塞，若无用于读取的熵，则需调用函数 `schedule()` 来释放处理器的控制权。（变量 `entropy_count` 包含位信息而不是字节信息，因而可以通过除以 8 来判断熵池是否已满。）

第 1638~1639 行：当我们终于重新开始时，清除等待队列。

注意 Linux 中的 `random` 设备要求当熵池队列满时才返回一个随机数，但 `urandom` 设备并无这种要求，它返回时并不考虑熵池中可用数据的大小。

让我们进一步看看当某个任务调用函数 `schedule()` 时会执行哪些相关操作：

```

-----
kernel/sched.c
2184 asmlinkage void __sched schedule(void)
2185 {
...
2209     prev = current;

```

```

...
2233  switch_count = &prev->nivcsw;
2234  if (prev->state && !(preempt_count() & PREEMPT_ACTIVE)) {
2235      switch_count = &prev->nvcsw;
2236      if (unlikely((prev->state & TASK_INTERRUPTIBLE) &&
2237          unlikely(signal_pending(prev))))
2238          prev->state = TASK_RUNNING;
2239      else
2240          deactivate_task(prev, rq);
2241  }
2242  ...

```

第 2209 行：变量 `prev` 中存储的是一个指针，指向当前任务的任务结构。当该任务自己调用函数 `schedule()` 时，指针 `current` 就会指向它的结构体。

第 2233 行：我们把任务的上下文切换计数器 `nivcsw` 存放在 `switch_count` 中，切换成功后 `nivcsw` 就会加 1^①。

第 2234 行：只有当任务的状态变量 `prev->state` 非零且不存在内核抢占时，才执行该条件语句。换句话说，当任务的状态不是 `TASK_RUNNING` 且内核并未抢占该任务时，才执行该语句。

第 2235~2241 行：如果该任务是可中断的，那么它肯定会交出处理器的控制权。若应该释放控制权的任务还有挂起的信号，则将其状态设置为 `TASK_RUNNING`，以便向其他任务传递控制权之后，这个任务仍然有机会再次被调度程序选中投入运行。若无挂起的信号（这是普遍情况），则使该任务处于非活动状态，并将 `switch_count` 赋给 `nvcsw`。随后，调度程序将 `switch_count` 的值加 1，因此 `nvcsw` 或 `nivcsw` 也进行了加 1 操作。

然后，函数 `schedule()` 在调度程序的运行队列中选出另一个任务，并把控制权交给它^②。

如果当前任务由于某种原因需要等待，可以调用函数 `schedule()` 将处理器的控制权交给另一个内核任务。内核中的其他任务就利用这个机会获得处理器来完成自身的工作，当控制权交还给调用 `schedule()` 函数的任务时，导致该任务等待的原因已被移除。

再回到函数 `random_read()` 上来。最后，内核把控制权又交给了 `random_read()` 函数，然后清空等待队列，并继续执行。这将重复前面的操作，如果系统中已产生足够的熵，我们就能返回所请求的随机字节数。

`random_read()` 函数在调用函数 `schedule()` 前将自己的状态设置为 `TASK_INTERRUPTIBLE`，以便处于等待队列时可以通过信号来唤醒。调用函数 `batch_entropy_process()` 和 `random_ioctl()` 中的 `wake_up_interruptible()`，可以收集额外的熵，此时驱动程序自身的代码就会产生这些可唤醒自己的信号。通常，若该任务等待硬件的响应而不是软件的响应，其状态被设置为 `TASK_UNINTERRUPTIBLE`（反之则被设置为 `TASK_INTERRUPTIBLE`）。

函数 `random_read()` 用于将控制权传给其他任务，实现这部分功能的代码（参见 `drivers/char/random.c` 中的第 1632~1639 行）源自调度程序代码中函数 `interruptible_sleep_on()` 的变体。

① 如何使用上下文切换计数器可参阅第 4 章和第 7 章。

② 如要了解详细信息，可参阅第 7 章中的“`switch_to()`”部分。

```

-----
kernel/sched.c
2489 #define SLEEP_ON_VAR          \
2490     unsigned long flags;        \
2491     wait_queue_t wait;          \
2492     init_waitqueue_entry(&wait, current);
2493
2494 #define SLEEP_ON_HEAD          \
2495     spin_lock_irqsave(&q->lock, flags); \
2496     __add_wait_queue(q, &wait); \
2497     spin_unlock(&q->lock);
2498
2499 #define SLEEP_ON_TAIL          \
2500     spin_lock_irq(&q->lock); \
2501     __remove_wait_queue(q, &wait); \
2502     spin_unlock_irqrestore(&q->lock, flags);
2503
2504 void fastcall __sched interruptible_sleep_on(wait_queue_head_t *q)
2505 {
2506     SLEEP_ON_VAR
2507
2508     current->state = TASK_INTERRUPTIBLE;
2509
2510     SLEEP_ON_HEAD
2511     schedule();
2512     SLEEP_ON_TAIL
2513 }
-----

```

`q` 是一个 `wait_queue_head` 型数据结构，用于调整模块的休眠和等待状态。

第 2494~2497 行：自动将任务添加到等待队列 `q` 中。

第 2499~2502 行：自动从等待队列 `q` 中删除任务。

第 2504~2513 行：添加到等待队列，并将处理器的控制权交给另一任务。当获得对处理器的控制权后，从队列中删除自身。

函数 `random_read()` 使用自己的等待队列代码而不是标准宏，但本质上还是使用 `interruptible_sleep_on()` 函数来处理异常，只要 `random_read()` 函数还有多个可用的熵，那它就不会让出处理器，而是再次循环以便获取所有请求的熵。但若没有足够的熵，`random_read()` 就会处于等待状态，直到驱动程序获取了可使用的熵之后，由 `wake_up_interruptible()` 函数将它唤醒。

10.1.5 工作队列及中断

Linux 中的设备驱动程序都须例行公事地处理由与之交互的设备产生的中断。中断可以触发设备驱动程序中的中断处理程序，并暂停当前用户空间和内核空间正在执行的代码。显然，驱动程序中的中断处理程序执行得越快越好，以免长时间占用内核。

尽管如此，我们还是遇到了中断处理中标准的难题：如何处理一项任务繁重的中断呢？通常的做法是使用上半部例程和下半部例程。上半部例程可以快速接收中断并调度下半部例程，而由下半部例程来完成主要工作，下半部分一有机会就被执行。通常，上半部例程在执行时禁用中断，以确保一个中断处理程序不会被同一中断再次中断，因而设备驱动程序不必处理递归中断。下半

部例程通常在启用中断的情况下运行，以便在它继续执行大部分工作时处理其他中断。

在 Linux 以前的版本中，任务队列划分的上半部分和下半部分也被称为快速中断和慢速中断。Linux 2.6 内核中引入了工作队列的概念，现在工作队列已是处理下半部中断的标准途径。

当内核接收到中断时，处理器应当停止当前任务的运行，以便立即处理中断。当 CPU 进入这一模式时，通常就说它处于中断上下文中。此时，内核确定要将控制权交给哪个中断处理程序。当设备驱动程序想处理中断时，它将使用函数 `request_irq()` 来请求该中断号，并在该中断发生时注册被调用的中断处理函数，该操作通常在模块初始化时进行。由 `request_irq()` 注册的上半部中断处理函数只完成最基本的管理工作，然后将余下的部分以任务的形式插入工作队列，以便等候处理。

和上半部机制中使用的 `request_irq()` 函数一样，工作队列通常也是在模块初始化时注册的。它们可以通过宏 `DECLARE_WORK()` 静态初始化，也可以调用 `INIT_WORK()` 来分配任务结构体，并进行动态初始化。下面是对这些宏的定义：

```
-----
include/linux/workqueue.h
30 #define DECLARE_WORK(n, f, d) \
31     struct work_struct n = __WORK_INITIALIZER(n, f, d)
...
45 #define INIT_WORK(_work, _func, _data) \
46     do { \
47         INIT_LIST_HEAD(&(_work)->entry); \
48         (_work)->pending = 0; \
49         PREPARE_WORK((_work), (_func), (_data)); \
50         init_timer(&(_work)->timer); \
51     } while (0)
-----
```

上述两种宏均使用了如下参数。

- **n** 或者 **work**: 创建或初始化的任务结构体名称。
- **f** 或者 **func**: 当从工作队列中删除一个任务结构体时所执行的函数。
- **d** 或者 **data**: 用于保存函数 **f** 或 **func** 运行时传递给它们的数据。

在函数 `register_irq()` 中注册的中断处理函数用于接收中断，并将相关数据从中断处理程序的上半部分传递给其下半部分，这是通过设置数据结构 `work_struct` 并调用工作队列中的 `schedule_work()` 函数来实现的。

工作队列函数中的代码可在进程上下文中执行，并可完成在中断上下文中不可能做的事情，例如向用户空间写入数据，复制用户空间的数据以及睡眠等。

Tasklet 和工作队列相似，但完全在中断上下文中运行。当下半部的工作量很少时，Tasklet 大有用处，它能够节省中断处理函数中上半部和下半部的开销。宏 `DECLARE_TASKLET()` 可以初始化 Tasklet。

```
-----
include/linux/interrupt.h
136 #define DECLARE_TASKLET(name, func, data) \
137 struct tasklet_struct name = { NULL, 0, ATOMIC_INIT(0), func, data }
-----
```

- **name**: 创建的 tasklet 结构体的名称。
- **func**: 调度 tasklet 时需要调用的函数。
- **data**: 用于保存执行 tasklet 时传给函数 func 的数据。

要调度 tasklet, 需要使用函数 `tasklet_schedule()`:

```
-----
include/linux/interrupt.h
171 extern void FASTCALL(__tasklet_schedule(struct tasklet_struct *t));
172
173 static inline void tasklet_schedule(struct tasklet_struct *t)
174 {
175     if (!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
176         __tasklet_schedule(t);
177 }
-----
```

- **tasklet_struct**: 通过 `DECLARE_TASKLET()` 创建的 tasklet 的名称。

在上半部中断处理程序中, 可以调用函数 `tasklet_schedule()`, 以确保将来某一时刻, 在 tasklet 中声明的函数一定会被执行。tasklet 与工作队列的不同之处在于, 不同的 tasklet 可以同时不同的 CPU 上执行。若已调度的 tasklet 在执行前被再次被调度, 那么该 tasklet 仅执行一次。当 tasklet 在中断上下文中运行时, 它不能休眠也不能向用户空间复制数据。因为 tasklet 运行在中断上下文中, 如果不同的 tasklet 需要通信, 唯一安全的同步方式是使用自旋锁。

10.1.6 系统调用

除了通过设备驱动程序向内核添加代码外, 还有别的方式。在 Linux 内核系统中, 用户空间的程序可以通过系统调用来访问内核服务程序和系统硬件。用户态程序可用的许多 C 库例程都把一些代码以及一个或多个系统调用打包在一起, 以共同实现一个函数的功能。事实上, 通过内核代码也可以访问系统调用。

系统调用天生就与硬件相关。在 Intel 体系结构中, 所有系统调用都使用软中断 0x80。系统调用的参数都通过通用寄存器来传递, 在 x86 体系结构中, 系统调用最多可有 5 个参数。若需要 5 个以上的参数, 就只能传递一个指向参数块的指针。执行汇编指令 `int 0x80` 时, 处理器借助其异常处理能力来调用一个特定的内核态例程。

10.1.7 其他类型的驱动程序

迄今为止, 我们讨论的都是字符设备驱动程序, 它常常是最容易理解的, 你可能想编写一些以其他方式与内核打交道的驱动程序。

块设备与字符设备的相似之处在于, 通过文件系统就可以访问它们。系统中第一个 IDE 硬盘驱动器的设备文件是 `/dev/hda`。块设备注册和注销的方式与字符设备类似, 也是通过函数 `register_blkdev()` 和 `unregister_blkdev()` 来实现的。

块设备驱动程序与字符设备驱动程序的不同之处主要在于, 块设备驱动程序本身没有读写功能, 它们使用的是请求的方法。

2.6 版本的内核对块设备子系统做出了很大改动。以前的函数如 `block_read()` 和 `block_write()`，以及 `blk_size` 和 `blksize_size` 等内核数据结构都被删除了。本节仅着眼于 2.6 版本中块设备驱动程序的实现。

如果你的 Linux 内核必需支持磁盘设备（或类磁盘设备）。那么你就得编写一个块设备驱动程序，这个驱动程序必须告诉内核它将与哪种磁盘打交道。这就需要使用结构 `gendisk`：

```
-----
include/linux/genhd.h
82 struct gendisk {
83     int major;          /* major number of driver */
84     int first_minor;
85     int minors;
86     char disk_name[32]; /* name of major driver */
87     struct hd_struct **part; /* [indexed by minor] */
88     struct block_device_operations *fops;
89     struct request_queue *queue;
90     void *private_data;
91     sector_t capacity;
...
-----
```

第 83 行：`major` 是块设备的主设备号。可以静态设置它，也可以由 `register_blkdev()` 函数动态生成，这与字符设备完全一样。

第 84~85 行：`first_minor` 和 `minors` 用于确定块设备中的分区数，其中 `minors` 包含块设备中从设备号的最大值，`first_minor` 则是它的第一个从设备号。

第 86 行：`disk_name` 是一个 32 字符的块设备名，我们可以在文件系统 `/dev`、`sysfs` 和 `/proc/partitions` 中看到。

第 87 行：`hd_struct` 存放与该块设备相关的分区设置。

第 88 行：`fops` 是一个指向结构体 `block_operations` 的指针，该结构体包含操作 `open`、`release`、`ioctl`、`media`、`_changed` 以及 `revalidate_disk`（可参阅 `include/linux/fs.h` 文件）。在 2.6 版的内核中，每个设备都有自己的操作集。

第 89 行：指针 `request_queue` 指向一个队列，该队列可帮助管理设备中挂起的操作。

第 90 行：`private_data` 指向私有信息，内核的块设备子系统不能对其进行访问，其常见的应用是存放数据，这些数据用于低级的、与具体设备相关的操作。

第 91 行：`capacity` 描述以 512 字节扇区为单位的块设备大小。如果该设备是可移动的，如软盘和光盘，当 `capacity` 等于 0 时表示没有该设备。如果你的设备不是每扇区包含 512 字节，你仍然要这样设置。例如，如果你的设备使用了 1000 个 256 字节的扇区，那就相当于 500 个 512 字节的扇区。

除了需要 `gendisk` 数据结构外，块设备还需要使用自旋锁结构，以便与请求队列配合。

不论是自旋锁还是 `gendisk` 结构体中的字段都必须由设备驱动程序来初始化（登录 http://en.wikipedia.org/wiki/Ram_disk 可以获取有关初始化 RAM 磁盘块设备驱动程序的示例）。块设备初始化完毕并准备处理请求时，应当调用函数 `add_disk()` 将该设备添加到系统中。

最后，如果该块设备可以作为系统的熵源时，模块的初始化也可以调用函数 `add_disk_`

randomness() 来实现 (详情可查阅文件 drivers/char/random.c)。

既然已经了解块设备驱动程序初始化的基本知识, 那么我们就可以来看看它的全貌以及退出和清除该驱动程序等工作, 这在 Linux 2.6 中很容易实现。

del_gendisk (struct gendisk) 可将 gendisk 从系统中删除, 并清除其分区信息。但执行该调用之前必须先执行 putdisk (struct gendisk) 调用, 它用于释放内核中涉及 gendisk 的内容。可调用 unregister_blkdev(int major, char[16] device_name) 来注销块设备, 该函数允许我们释放 gendisk 结构。

我们也需要清除与块设备驱动程序相关的请求队列, 这可以使用系统调用 blk_cleanup_queue(struct*request_queue) 来实现。注意, 若只能通过 gendisk 来引用请求队列, 则应当在释放 gendisk 前调用 blk_cleanup_queue。

在对块设备的初始化和关闭进行概要说明时, 我们很容易不谈及有关请求队列的细节问题, 但既然已经建立了一个设备驱动程序, 那就必须实实在在地做点什么, 而请求队列就是完成块设备主要读写功能的部分。

```
-----
include/linux/blkdev.h
576 extern request_queue_t *blk_init_queue(request_fn_proc *, spinlock_t *);
...
-----
```

第 576 行: 为了创建请求队列, 需要使用 blk_init_queue, 并传给它一个指向自旋锁的指针和一个指向请求函数的指针, 其中自旋锁可控制对队列的访问, 而任何时候访问该设备都必须调用这个请求函数。该请求函数的原型如下:

```
static void my_request_function( request_queue_t *q );
```

通常, 任务繁重的请求函数可以很方便地使用许多辅助函数。为了确定下一个要处理的请求, 应当调用函数 elv_next_request(), 它将返回一个指向请求结构体的指针, 没有下一个请求时该函数返回 NULL。

在 Linux 2.6 内核中, 块设备驱动程序在请求结构体中遍历所有的 BIO 结构。BIO 代表块设备 I/O, include/linux/bio.h 中有它的完整定义。

BIO 结构包含一个指向 biovec 结构体链表的指针, 该结构体定义如下:

```
-----
include/linux/bio.h
47 struct bio_vec {
48     struct page *bv_page;
49     unsigned int bv_len;
50     unsigned int bv_offset;
51 };
-----
```

每个 biovec 都使用其页面结构来存放数据缓冲区, 缓冲区中的数据是从磁盘读出的或最终将写入磁盘。Linux 2.6 内核中有大量 bio 辅助例程, 可用于反复使用 bio 结构中的数据。

为了确定 BIO 操作的大小, 可以查看 BIO 结构中的 bio_size 字段, 得到的结果以字节为单

位,也可以使用 `bio_sectors()` 宏,得到以扇区来计算 BIO 操作的大小。块操作的类型,如 READ 或 WRITE,可由 `bio_data_dir()` 决定。

为了遍历 BIO 结构中的 `biovec` 链表,可以使用 `bio_for_each_segment()` 宏。在该循环中,可以使用更多的宏,以便深入研究 `biovec_bio_page()`、`bio_offset()`、`bio_curr_sectors()` 和 `bio_data()`。详情可查阅 `include/linux/bio.h` 和 `Documentation/block/biodoc.txt`。

包含在 `biovec` 和页面结构中的某些字段组合起来有助于确定应当向块设备读写什么数据,而如何读写设备这些更底层的细节则依赖于块设备驱动程序所使用的硬件。

既然已经明了该如何遍历 BIO 结构,那么就必须弄清楚该如何遍历 BIO 结构中的请求结构体链表。这需要用到另外一个宏 `rq_for_each_bio`:

```
-----
include/linux/blkdev.h
495 #define rq_for_each_bio(_bio, rq) \
496     if ((rq->bio)) \
497         for (_bio = (rq->bio); _bio; _bio = bio->bi_next)
-----
```

第 495 行: `bio` 是指当前 BIO 结构, `rq` 是要循环遍历的请求。

每个 BIO 都处理过之后,驱动程序就该更新内核。这一操作是使用函数 `end_that_request_first()` 来完成的。

```
-----
include/linux/blkdev.h
557 extern int end_that_request_first(struct request *, int, int);
-----
```

第 557 行:不出错时,第一个 `int` 参数应当非零,第二个 `int` 参数代表该设备已处理的扇区数。

当 `end_that_request_first()` 函数返回 0 时,所有请求都已处理完毕,可以开始进行清除工作了。这可以调用函数 `blkdev_dequeue_request()` 和 `end_that_request_last()` 来完成,按照这个顺序,这两个函数都把请求当做唯一的参数进行处理。

这时,请求函数已经完成它的工作。接下来,块设备子系统使用块设备驱动程序的请求队列函数来执行磁盘操作。该设备也许还要处理某些 `ioctl` 函数,如处理 RAM 磁盘分区的函数,但这还得取决于块设备的类型。

本节仅略述了块设备的一些基本知识,但 DMA 操作函数、聚集、请求队列命令预操作,以及更先进的块设备的许多特性都没有介绍。要想进一步了解有关信息,可以参阅 `Documentation/block` 目录下的文件。

10.1.8 设备模型和 `sysfs` 文件系统

Linux 2.6 内核的新颖之处在于 Linux 设备模型,这是与 `sysfs` 密切相关的。设备模型存放着与系统中设备及驱动程序相关的一组内部数据。`sysfs` 系统还记录现有的设备,并将其分类:块设备、输入设备、总线设备,等等。系统也记录已有的设备驱动程序以及它们如何与所管理的设备关联。设备模型处于内核内部,而 `sysfs` 就是通向该模型的窗口。由于某些设备和驱动程序

并不通过 sysfs 公开自身，因此最好将 sysfs 理解为内核设备模型的公共视图。

某些设备在 sysfs 中有多项。

尽管设备模型中只有一份数据副本，但正如我们在 sysfs 树中看到的符号链接一样，可以通过多种方式来访问这些数据。

sysfs 文件的层次结构与内核的 kobject 和 kset 结构有关。该模型相当复杂，但绝大多数驱动程序的开发人员不必深究其细节就能完成许多有用的任务^①。当利用 sysfs 属性的概念时，就涉及 kobject，但这是一种抽象的方式。属性是设备或驱动程序模型的组成部分，可以通过 sysfs 文件系统对其访问或修改。属性可能是内部模块变量，用于控制模块对任务的管理，也可能被直接链接从而对各种硬件进行设置。例如，RF 发送器，它工作时有一个基本频率，根据这个基本频率稍作变动就可以实现多个调谐器。我们可以将 RF 驱动程序的模块属性对 sysfs 公开，来改变它的基本频率。

访问某一属性时，sysfs 将调用一个函数来处理该操作。show() 函数用于读取属性，store() 函数用于写入一个属性。show() 和 store() 函数可接收的数据大小最多为一页。

大致了解 sysfs 的工作原理后，可进一步探讨驱动程序通过 sysfs 来注册的细节，公开一些属性，当访问这些属性时，注册要操作的具体函数 show() 和 store()。

我们的首要任务是确定新设备及其驱动程序的类型（例如，usb_device、net_device、pci_device、sys_device 等）。所有这些设备的结构体中都有一个 char*name 字段，sysfs 将使用该字段名在 sysfs 层次结构中标明该设备。

分配完设备结构并为其命名后，就必须创建并初始化 devicer_driver 结构：

```
-----
include/linux/device.h
102 struct device_driver {
103     char      * name;
104     struct bus_type * bus;
105
106     struct semaphore unload_sem;
107     struct kobject kobj;
108     struct list_head devices;
109
110     int (*probe) (struct device * dev);
111     int (*remove) (struct device * dev);
112     void (*shutdown) (struct device * dev);
113     int (*suspend) (struct device * dev, u32 state, u32 level);
114     int (*resume) (struct device * dev, u32 level);
115};
-----
```

第 103 行：name 表示在 sysfs 层次结构中显示的设备驱动程序名。

第 104 行：bus 通常是被自动填充的，驱动程序的编写者不用考虑它。

第 105~115 行：程序员无需设置其他字段的值，它们应当在总线级自动被初始化。

在初始化的过程中，可以调用 driver_register() 来注册设备驱动程序，该函数会把大部

① 可参阅内核源代码中的文件 documentation/filesystems/sysfs.txt。

分工作交给 `bus_add_driver()` 去做。类似地，设备驱动程序退出时，要调用 `driver_unregister()` 进行注销。

```
-----
drivers/base/driver.c
86 int driver_register(struct device_driver * drv)
87 {
88     INIT_LIST_HEAD(&drv->devices);
89     init_MUTEX_LOCKED(&drv->unload_sem);
90     return bus_add_driver(drv);
91 }
-----
```

注册设备驱动程序之后，可以通过结构 `driver_attribute` 和十分有用的宏 `DRIVER_ATTR` 为驱动程序设置属性：

```
-----
include/linux/device.h
133 #define DRIVER_ATTR(_name, _mode, _show, _store) \
134 struct driver_attribute driver_attr_##_name = { \
135     .attr = {.name = __stringify(_name), .mode = _mode, .owner = THIS_MODULE \
136     }, \
137     .show = _show, \
138     .store = _store, \
139 };
-----
```

第135行：`name` 是设备驱动程序属性的名称，`mode` 是描述属性保护级别的位图。在文件 `include/linux/stat.h` 中包含许多这样的模式，`S_IRUGO`（用于只读操作）和 `S_IWUSR`（用于根用户的写访问）仅是其中的两例。

第136行：`show` 是通过 `sysfs` 读取其属性时所使用的驱动程序函数名。若禁止读取设备的属性，则将 `show` 赋值为 `NULL`。

第137行：`store` 是通过 `sysfs` 写入其属性时所使用的驱动程序的函数名。若禁止写入设备的属性，则将 `store` 赋值为 `NULL`。

为具体的驱动程序实现 `show()` 和 `store()` 的驱动程序函数必须遵循以下函数原型：

```
-----
include/linux/sysfs.h
34 struct sysfs_ops {
35     ssize_t (*show)(struct kobject *, struct attribute *, char *);
36     ssize_t (*store)(struct kobject *, struct attribute *, const char *, size_t);
37 };
-----
```

回想一下，前面我们说到对 `sysfs` 属性进行读写的限制是 `PAGE_SIZE` 字节。所以驱动程序属性函数 `show()` 和 `store()` 应当确保强制遵守该限制。

现在，可以为内核设备驱动程序添加基本的 `sysfs` 功能了。要想进一步了解有关 `sysfs` 及 `kobject` 的信息，可以查看 `Documentation/device-model` 目录下的文件。

另一种设备驱动程序类型是网络设备驱动程序。网络设备可以发送和接收数据包，它不一定

是一个硬件设备，环回设备就是一个软件网络设备。

10.2 编写代码

10.2.1 设备基础

当你编写了一个设备驱动程序时，该程序通过文件系统中的入口与操作系统进行交互。该入口包含一个主设备号，以便告诉内核访问文件时应当使用哪个驱动程序；同时还有一个从设备号，用于扩大该设备驱动程序的使用范围。加载该设备驱动程序之后，它会注册其主设备号。可以通过查看文件 `/proc/devices` 来获取注册信息：

```
-----
lkp# less /proc/devices
Character devices:
 1 mem
 2 pty
 3 tty
 4 ttyS
 5 cua
 6 lp
 7 vcs
10 misc
29 fb
128 ptm
136 pts
Block devices:
 1 ramdisk
 2 fd
 3 ide0
 7 loop
22 ide1
-----
```

当设备驱动程序向内核注册自己时，其主设备号就会被写入 `/proc/devices` 文件；对于每个字符设备，这一过程都是调用 `register_chrdev()` 函数来实现的。

```
-----
include/linux/fs.h
1: int register_chrdev(unsigned int major, const char *name,
2:      struct file_operations *fops)
-----
```

- **major**: 已注册设备的主设备号。如果 `major` 等于 0，那么内核会动态地为它分配一个与其他已加载的模块不冲突的主设备号。
- **name**: 该字符串表示文件系统 `/dev` 目录下的设备。
- **fops**: 指向 `file-operations` 结构体的指针，它定义了对已注册设备可进行的操作。

如果在创建设备时未设置其主设备号，使用 0 作为主设备号是首选的方法（IDE 驱动程序的主设备号总是为 3，SCSI 为 8，floppy 为 2）。此时需要动态地为它分配一个主设备号，以避免人

为地分配一些可能已使用的主设备号^①。但这样会使得文件系统节点变得稍微复杂一些，因为加载模块后，必须检查该设备的主设备号。例如，检测一个设备时，可能需要进行如下操作：

```
-----
lkp@lcp# insmod my_module.o
lkp@lcp# less /proc/devices
1 mem
...
233 my_module
lkp@lcp# mknod c /dev/my_module0 233 0
lkp@lcp# mknod c /dev/my_module1 233 1
-----
```

这些代码告诉我们如何使用 `insmod` 命令来插入模块。 `insmod` 命令可在内核运行时安装可加载的模块。该模块的代码包括：

```
-----
static int my_module_major=0;
...
module_param(my_module_major, int, 0);
...
result = register_chrdev(my_module_major, "my_module", &my_module_fops);
-----
```

前面两行告诉我们如何创建一个值为 0 的默认主设备号，以便需要时进行动态分配，但它允许用户使用变量 `my_module_major` 作为模块的参数，因而不必考虑动态分配。

```
-----
include/linux/moduleparam.h
1: /* This is the fundamental function for registering boot/module
   parameters. perm sets the visibility in driverfs: 000 means it's
   not there, read bits mean it's readable, write bits mean it's
   writable. */
...
/* Helper functions: type is byte, short, ushort, int, uint, long,
   ulong, charp, bool or invbool, or XXX if you define param_get_XXX,
   param_set_XXX and param_check_XXX. */
...
2: #define module_param(name, type, perm)
-----
```

在 Linux 以前的版本中，宏 `module_param` 就是宏 `MODULE_PARM`。Linux 2.6 反对这种做法，它要求必须使用 `module_param`。

- ❑ **name**：该字符串用于访问参数的值。
- ❑ **type**：存放在参数名中的值的类型。
- ❑ **perm**：在 `sysfs` 中可以看到的模块参数名。如果不知道 `sysfs` 是什么，就使用 0，表示不能通过 `sysfs.perm` 访问该参数。

^① `register_chrdev()` 函数返回已分配的主设备号。若要为某设备动态分配主设备号，`register_chrdev()` 函数也许在获取该设备号时大有所为。

回想一下，我们曾经传给 `register_chrdev()` 函数一个指向 `fops` 数据结构的指针。它用于告诉内核驱动程序可以处理什么函数。我们只声明那些模块处理的函数。为了在注册的设备中将 `read`、`write`、`ioctl`、`open` 声明为有效操作，需要添加如下代码：

```
-----
struct file_operations my_mod_fops = {
    .read = my_mod_read,
    .write = my_mod_write,
    .ioctl = my_mod_ioctl,
    .open = my_mod_open,
};
-----
```

10.2.2 符号输出

编写一个复杂的设备驱动程序时，也许要输出驱动程序中定义的某些符号，以便让内核中其他模块使用。这通常用于低级的驱动程序，以便根据这些基础功能构建更高级的驱动程序。

任何设备驱动程序加载后，其所有已输出的符号都会存放内核符号表中，而随后加载的设备驱动程序就可以使用这些由先前的驱动程序输出的符号。当模块之间相互依赖时，加载的顺序就显得至关重要。如果高层模块所需的符号不存在的话，`insmod` 操作就会失败。

在 Linux 2.6 内核中，设备程序员可用如下两个宏来输出符号：

```
-----
include/linux/module.h
187 #define EXPORT_SYMBOL(sym)          \
188     __EXPORT_SYMBOL(sym, "")
189
190 #define EXPORT_SYMBOL_GPL(sym)        \
191     __EXPORT_SYMBOL(sym, "_gpl")
-----
```

使用宏 `EXPORT_SYMBOL` 将符号输出到内核符号表时，允许内核中的任意模块使用这些给定的符号，但使用宏 `EXPORT_SYMBOL_GPL` 时，只有当模块的 `MODULE_LICENSE` 属性中定义了 GPL 兼容许可时，才能够使用这些符号。（该许可的完整列表可以参阅文件 `include/linux/module.h`。）

10.2.3 IOCTL

迄今为止，我们研究的设备驱动程序都是主动操作，或向设备进行数据的读写操作。那么，当一个设备驱动程序不只具有读写功能时，会怎么样呢？若设备驱动程序可以进行不同类型的读写操作时又会怎样呢？又或者当设备需要某种硬件控制接口时会怎样呢？在 Linux 中，设备驱动程序解决这些问题的常用方式就是使用 `ioctl`。

`ioctl` 是一个系统调用，它允许设备驱动程序处理用于控制 I/O 通道的特殊命令。设备驱动程序的 `ioctl` 调用必须遵循 `file_operations` 结构中的如下声明：

```
-----
include/linux/fs.h
863 struct file_operations {
-----
```

```
...
872 int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
-----
```

在用户空间中，ioctl 函数调用定义如下：

```
int ioctl (int d, int request, ...);
```

用户空间定义的第三个参数对于内存来说是一个无类型的指针，这就是把数据从用户空间传递给设备驱动程序的 ioctl 实现的方法。也许看上去很复杂，但在驱动程序中实际应用 ioctl 时相当简单。

首先，为设备声明有效的 IOCTL 号。我们可以参阅文件 Documentation/ioctl-number.txt 并选择一段机器不会使用的代码。查阅当前的 2.6 文件，可以看到 ioctl 代码中字符 'g' 没有被使用。所以，在我们的设备驱动程序中，我们这样声明该字符：

```
#define MYDRIVER_IOC_MAGIC 'g'
```

对于每个设备驱动程序接收到的独特的控制信息，我们都要为它声明一个特有的 ioctl 号。它并不是基于已定义的魔数：

```
-----
#define MYDRIVER_IOC_OP1 _IO(MYDRIVER_IOC_MAGIC, 0)
#define MYDRIVER_IOC_OP2 _IOW(MYDRIVER_IOC_MAGIC, 1)
#define MYDRIVER_IOC_OP3 _IOW(MYDRIVER_IOC_MAGIC, 2)
#define MYDRIVER_IOC_OP4 _IORW(MYDRIVER_IOC_MAGIC, 3)
-----
```

使用 include/asm/ioctl.h 文件中定义的宏 MYDRIVER_IOC_MAGIC 给已列出的四种操作 (op1、op2、op3 以及 op4) 分别分配一个唯一的 ioctl 号。其中宏 MYDRIVER_IOC_MAGIC 是一个 ioctl 魔数。该文档说明了所有信息的含义：

```
-----
Documentation/liioctl-number.txt
6 If you are adding new ioctls to the kernel, you should use the _IO
7 macros defined in <linux/ioctl.h>:
8
9 _IO an ioctl with no parameters
10 _IOW an ioctl with write parameters (copy_from_user)
11 _IOR an ioctl with read parameters (copy_to_user)
12 _IORW an ioctl with both write and read parameters.
13
14 'Write' and 'read' are from the user's point of view, just like the
15 system calls 'write' and 'read'. For example, a SET_FOO ioctl would
16 be _IOW, although the kernel would actually read data from user space;
17 a GET_FOO ioctl would be _IOR, although the kernel would actually write
18 data to user space.
-----
```

从用户空间可以像下面这样调用 ioctl 命令：

```
-----
ioctl(fd, MYDRIVER_IOC_OP1, NULL);
ioctl(fd, MYDRIVER_IOC_OP2, &mydata);
```

```
ioctl(fd, MYDRIVER_IOC_OP3, mydata);
ioctl(fd, MYDRIVER_IOC_OP4, &mystruct);
```

用户空间的程序必需知道 `ioctl` 命令是什么（本例中有 `MYDRIVER_IOC_OP1` 到 `MY_DRIVER_IOC_OP4`）以及这些命令需要什么类型的参数。我们可以通过使用 `ioctl` 系统调用的返回代码来返回一个值，或者将这个参数当作一个需要设置或读入的指针进行解释。对于后一种情况，要注意的一点是，使用指针会涉及将用户空间内存的字节移入/移出内核空间的操作。

使用 `ioctl` 函数来实现内存存在用户空间和内核空间的交换，最简单的方法就是使用 `put_user()` 以及 `get_user()` 函数，这些函数的定义如下：

```
-----
#include/asm-i386/uaccess.h
* get_user: - Get a simple variable from user space.
* @x: Variable to store result.
* @ptr: Source address, in user space.
...
* put_user: - Write a simple value into user space.
* @x: Value to copy to user space.
* @ptr: Destination address, in user space.
-----
```

函数 `put_user()` 和 `get_user()` 确保要读取或写入用户空间的内存，在调用这些函数时便已在内存中。

也许你还想为设备驱动程序的 `ioctl` 函数添加另一个约束条件，那就是身份验证。

检查调用 `ioctl` 函数的进程是否具有调用 `ioctl` 的权限，通常使用权能。驱动程序身份验证中常用的权能是 `CAP_SYS_ADMIN`：

```
-----
include/linux/capability.h
202 /* Allow configuration of the secure attention key */
203 /* Allow administration of the random device */
204 /* Allow examination and configuration of disk quotas */
205 /* Allow configuring the kernel's syslog (printk behavior) */
206 /* Allow setting the domainname */
207 /* Allow setting the hostname */
208 /* Allow calling bdflush() */
209 /* Allow mount() and umount(), setting up new smb connection */
210 /* Allow some autofs root ioctls */
211 /* Allow nfsservctl */
212 /* Allow VM86_REQUEST_IRQ */
213 /* Allow to read/write pci config on alpha */
214 /* Allow irix_prctl on mips (setstacksize) */
215 /* Allow flushing all cache on m68k (sys_cacheflush) */
216 /* Allow removing semaphores */
217 /* Used instead of CAP_CHOWN to "chown" IPC message queues, semaphores
218 and shared memory */
219 /* Allow locking/unlocking of shared memory segment */
220 /* Allow turning swap on/off */
221 /* Allow forged pids on socket credentials passing */
222 /* Allow setting readahead and flushing buffers on block devices */
-----
```



```

223 /* Allow setting geometry in floppy driver */
224 /* Allow turning DMA on/off in xd driver */
225 /* Allow administration of md devices (mostly the above, but some
226 extra ioctls) */
227 /* Allow tuning the ide driver */
228 /* Allow access to the nvram device */
229 /* Allow administration of apm_bios, serial and bttv (TV) device */
230 /* Allow manufacturer commands in isdn CAPI support driver */
231 /* Allow reading non-standardized portions of pci configuration space */
232 /* Allow DDI debug ioctl on sbpcd driver */
233 /* Allow setting up serial ports */
234 /* Allow sending raw qic-117 commands */
235 /* Allow enabling/disabling tagged queuing on SCSI controllers and sending
236 arbitrary SCSI commands */
237 /* Allow setting encryption key on loopback filesystem */
238
239 #define CAP_SYS_ADMIN 21
-----

```

include/linux/capability.h 中定义了许多其他更具体的权能，它们也许更适合权限要求更严的设备驱动程序，但 CAP_SYS_ADMIN 已包罗万象了。

为了在驱动程序中检查调用进程的权能，可以添加类似下面这样的代码：

```

if (! capable(CAP_SYS_ADMIN)) {
    return EPERM;
}

```

10.2.4 轮询与中断

当设备驱动程序向它控制的设备发出命令时，可以通过两种方式来判断该设备是否已经成功接收到命令，分别是设备轮询方式和设备中断方式。

轮询方式意味着设备驱动程序要周期性地检查设备的状态，以确保相应设备成功地接收命令。由于设备驱动程序是内核的一部分，如果直接轮询访问设备，可能会带来灾难性的后果，内核将在此过程中无所事事，直到设备完成轮询操作。设备驱动程序可以通过使用系统定时器来避免这种现象。当设备驱动程序要轮询访问设备时，它将在稍后调度内核，以调用设备驱动中的某个例程来检查设备状态，而无须让内核闲置。

深入探讨内核中断的工作原理之前，必须先来研究锁定访问内核临界区代码的主要方法：自旋锁。自旋锁的作用就是在进入临界区之前为一些特定的标志位设置一定的值，在退出临界区之后再复位这些标志位。当进程上下文不能阻塞在内核代码中就是这种情况，应该使用自旋锁，我们来看看 x86 及 PPC 体系结构中有关自旋锁的代码。

```

-----
include/asm-i386/spinlock.h
32 #define SPIN_LOCK_UNLOCKED (spinlock_t) { 1 SPINLOCK_MAGIC_INIT }
33
34 #define spin_lock_init(x) do { *(x) = SPIN_LOCK_UNLOCKED; } while(0)
...
43 #define spin_is_locked(x) (*(volatile signed char *)&(x)->lock) <= 0)

```

```

44 #define spin_unlock_wait(x)  do { barrier(); } while(spin_is_locked(x))

include/asm-ppc/spinlock.h
25 #define SPIN_LOCK_UNLOCKED  (spinlock_t) { 0 SPINLOCK_DEBUG_INIT }
26
27 #define spin_lock_init(x)  do { *(x) = SPIN_LOCK_UNLOCKED; } while(0)
28 #define spin_is_locked(x)  ((x)->lock != 0)
while(spin_is_locked(x))
29 #define spin_unlock_wait(x)  do { barrier(); } while(spin_is_locked(x))
-----

```

在 x86 平台下, 当未加锁时自旋锁的 flag 值实际为 1, 而在 PPC 平台下, 这时 flag 的值为 0。由此可见, 编写一个驱动程序时, 应当使用宏而不是原始值, 以确保平台间的兼容。

想要获得自旋锁的任务将在紧凑循环中不断检测该特殊标志的值, 直到该标志的值小于 0。所以我们说, 等待自旋锁的任务处于自旋状态。(可参阅这两个代码块中的函数 `spin_unlock_wait()`。)

当内核想要执行一段不受任何中断打扰的临界区代码时, 驱动程序通常会在处理中断时使用自旋锁。在 Linux 以前的版本中, 使用函数 `cli()` 和 `sti()` 来禁用和启用中断。从 2.5.28 版开始, 逐渐用自旋锁代替了这两个函数, 这种新的方式使用如下代码来保证执行临界区代码时不被中断:

```

-----
Documentation/cli-sti-removal.txt
1: spinlock_t driver_lock = SPIN_LOCK_UNLOCKED;
2: struct driver_data;
3:
4: irq_handler (...)
5: {
6: unsigned long flags;
7: ....
   spin_lock_irqsave(&driver_lock, flags);
8: ....
10: driver_data.finish = 1;
11: driver_data.new_work = 0;
12: ....
13: spin_unlock_irqrestore(&driver_lock, flags);
14: ....
15: }
16:
17: ...
18:
19: ioctl_func (...)
20: {
21: ...
22: spin_lock_irq(&driver_lock);
23: ...
24: driver_data.finish = 0;
25: driver_data.new_work = 2;
26: ...
27: spin_unlock_irq(&driver_lock);

```



```
28: ...
29: }
```

第 8 行：进入临界区之前，保存中断标表并对 `driver_lock` 加锁。

第 9~12 行：每次仅允许一个任务执行临界区代码。

第 27 行：临界区末尾。恢复中断状态并对 `driver_lock` 解锁。

使用 `spin_lock_irq_save()`（以及 `spin_lock_irq_restore()`）函数，可以确保在中断处理程序工作之前就禁止中断，并在中断处理程序完成工作后仍保持禁止中断的状态。

当使用 `ioctl_func()` 函数对 `driver_lock` 加锁之后，其他调用 `irq_handler()` 的调用者就会自旋。因此，必须确保 `ioctl_func()` 函数中的临界区代码尽快被执行，以保证上半部中断处理程序 `irq_handler()` 等待的时间尽可能短。

让我们来分析以下如何创建一个中断程序及其上半部中断处理程序（参阅 10.2.5 节中的下半部中断处理程序，它使用了工作队列）：

```
-----
#define mod_num_tries 3
static int irq = 0;
...
int count = 0;
unsigned int irqs = 0;
while ((count < mod_num_tries) && (irq <= 0)) {
    irqs = probe_irq_on();
    /* Cause device to trigger an interrupt.
       Some delay may be required to ensure receipt
       of the interrupt */
    irq = probe_irq_off(irqs);
    /* If irq < 0 multiple interrupts were received.
       If irq == 0 no interrupts were received. */
    count++;
}
if ((count == mod_num_tries) && (irq <= 0)) {
    printk("Couldn't determine interrupt for %s\n",
        MODULE_NAME);
}
-----
```

这些代码将会成为设备驱动程序初始化代码的一部分，在没有中断时会执行失败。既然我们有一个中断，那就可以在内核中注册该中断处理程序及其上半部中断处理程序。

```
-----
retval = request_irq(irq, irq_handler, SA_INTERRUPT,
    DEVICE_NAME, NULL);
if (retval < 0) {
    printk("Request of IRQ %n failed for %s\n",
        irq, MODULE_NAME);
    return retval;
}
-----
```

request_irq()函数的原型为:

```
-----
arch/ i386/kernel/irq.c
590 /**
591 * request_irq - allocate an interrupt line
592 * @irq: Interrupt line to allocate
593 * @handler: Function to be called when the IRQ occurs
594 * @irqflags: Interrupt type flags
595 * @devname: An ascii name for the claiming device
596 * @dev_id: A cookie passed back to the handler function
...
622 int request_irq(unsigned int irq,
623     irqreturn_t (*handler)(int, void *, struct pt_regs *),
624     unsigned long irqflags,
625     const char * devname,
626     void *dev_id)
```

irqflags 参数可以是下列宏的 ord 值。

- SA_SHIRQ: 表明该中断可由多个设备共享;
- SA_INTERRUPT: 运行 handler 时禁止本地中断;
- SA_SAMPLE_RANDOM: 表明该中断是否是熵源。

如果中断没有被共享,那么 dev_id 必须等于 NULL;反之,由于 handler 接收了其值,因此 dev_id 通常用来存放该设备地址的数据结构。

当模块退出时,应该使用 free_irq()函数来释放每一个请求的中断:

```
-----
arch/ i386/kernel/irq.c
669 /**
670 * free_irq - free an interrupt
671 * @irq: Interrupt line to free
672 * @dev_id: Device identity to free
...
682 */
683
684 void free_irq(unsigned int irq, void *dev_id)
```

如果 dev_id 是一个共享的 irq,那么模块就要确保在调用这个函数之前禁用中断。而且,在中断上下文中绝不允许调用 free_irq()函数。在模块的清除例程中调用 free_irq()函数是我们一贯的做法(参阅函数 spin_lock_irq()及 spin_unlock_irq())。

这时,我们已经注册了中断处理程序以及相关的 irq。现在就必须编写上半部中断处理程序了,我们把它定义为 irq_handler():

```
-----
void irq_handler(int irq, void *dev_id, struct pt_regs *regs)
{
    /* See above for spin lock code */
    /* Copy interrupt data to work queue data for handling in
```

```

    bottom-half */
    schedule_work( WORK_QUEUE );
    /* Release spin_lock */
}
-----

```

如果你仅仅需要一个快速中断处理程序，可以用 tasklet 来代替工作队列：

```

-----
void irq_handler(int irq, void *dev_id, struct pt_regs *regs)
{
    /* See above for spin lock code */
    /* Copy interrupt data to tasklet data */
    tasklet_schedule( TASKLET_QUEUE );
    /* Release spin_lock */
}
-----

```

10.2.5 工作队列和 tasklet

中断处理程序中的大多数任务都是由工作队列完成的。在上一节中，我们看到中断处理程序的上半部会将中断的相关数据从中断复制到一个数据结构中，之后再调用 `schedule_work()` 函数。

为了让任务从工作队列中运行，任务必须封装在 `work_struct` 结构中。为了在编译时就声明任务结构体，可以使用 `DECLARE_WORK()` 宏。例如，我们可以把下列代码添加到模块中，用相关的函数和数据初始化任务结构体。

```

-----
...
struct bh_data_struct {
    int data_one;
    int *data_array;
    char *data_text;
}
...
static bh_data_struct bh_data;
...
static DECLARE_WORK(my_mod_work, my_mod_bh, &bh_data);
...
static void my_mod_bh(void *data)
{
    struct bh_data_struct *bh_data = data;

    /* all the wonderful bottom half code */
}
-----

```

中断处理程序的上半部将通过 `my_mod_bh` 在 `bh_data` 中设置需要的所有数据，然后调用 `schedule_work(my_mod_work)` 函数。

`schedule_work()` 函数可用于任何模块，但这只是说，在通用的工作队列“事件”中我们会用到工作调度。某些模块可能想创建自己的工作队列，但是一个工作队列所需的函数只输出到

GPL 兼容的模块。因此，如果想保留模块的所有权的话，就必须使用通用的工作队列。

可以使用 `create_workqueue()` 宏来创建一个工作队列，该函数实际上是以第二个参数为 0 调用 `__create_workqueue()`。

```
-----
kernel/workqueue.c
304 struct workqueue_struct *__create_workqueue(const char *name,
305                                             int singlethread)
-----
```

`name` 可以多达 10 个字符。

如果 `singlethread` 等于 0，内核将为每个 CPU 创建一个 `workqueue` 线程；反之，若 `singlethread` 等于 1，内核仅为整个系统创建一个 `workqueue` 线程。

创建任务结构体的方式与前面介绍的类似，但将使用 `queue_work()` 函数而不是 `schedule_work()` 函数将它们置于自定义的工作队列中。

```
-----
kernel/workqueue.c
97 int fastcall queue_work(struct workqueue_struct *wq, struct work_struct
*work)
98 {
-----
```

`wq` 是通过 `create_workqueue()` 函数创建的自定义工作队列。

`work` 是添加到 `wq` 队列中的任务结构体。

我们可以在 `kernel/workqueue.c` 中看到如下其他工作队列函数：

- ❑ **queue_work_delayed()**。确保在指定 jiffies 数达到之前不调用该任务结构体函数。
- ❑ **flush_workqueue()**。使调用者等待，直到队列中所有被调度的任务都已完成。这通常在设备驱动程序退出时才使用。
- ❑ **destroy_workqueue()**。刷新并且释放工作队列。

类似的函数 `schedule_work_delayed()` 和 `flush_scheduled_work()` 用于通用的工作队列。

10.2.6 增加系统调用的代码

我们可以通过编辑 `/kernel` 目录下 `makefile` 文件来添加一个包含我们自己函数的文件，但更简单的方法是在源代码树中已经存在的文件内加入我们的函数代码。文件 `/kernel/sys.c` 中包含了用于系统调用的内核函数，文件 `arch/i386/kernel/sys_i386.c` 中以非标准的调用顺序给出了 x86 下的系统调用。我们可以将用 C 编写的系统调用函数的源代码添加到 `/kernel/sys.c` 文件中，这些代码将在内核态运行，并完成所有预定的工作。在这个过程中要做的其他事情就是支持用户使用该函数。通过 x86 的异常处理程序对函数进行分派。

```
-----
kernel/sys.c
1: ...
2: /* somewhere after last function */
-----
```



```

3:
4: /* simple function to demonstrate a syscall. */
5: /* take in a number, print it out, return the number+1 */
6:
7: asmlinkage long sys_ourcall(long num)
8: {
9: printk("Inside our syscall num =%d \n", num);
10: return(num+1);
11: }
-----

```

当异常处理程序处理 `int 0x80` 异常时，它就会在系统调用表中进行索引。在文件 `/arch/i386/kernel/entry.S` 中包含了底层的中断处理例程和系统调用表 `sys_call_table`。该系统调用表用汇编代码实现了 C 语言中的数组，表中每个元素都是 4 字节，并且都被初始化为一个函数地址。按照惯例，必须在这些函数的名字前加上前缀 `sys_`。因为该表中函数的位置决定了它的系统调用号，所以必须将我们的函数名添加到该表末尾。下面是添加修改后该表的内容：

```

-----
arch/i386/kernel/entry.S
: .data
608: ENTRY(sys_call_table)
    .long sys_restart_syscall /* 0 - old "setup()" system call, used for
restarting*/
...
    .long sys_tgkill /* 270 */
    .long sys_utimes
    .long sys_fadvise64_64
    .long sys_ni_syscall /* sys_vserver */
    .long sys_ourcall /* our syscall will be 274 */
884: nr_syscalls=(.-sys_call_table)/4
-----

```

文件 `include/asm/unistd.h` 将系统调用和它们在 `sys_call_table` 中的位置编号关联起来。而且该文件中的宏例程有助于用户程序（用 C 语言编写的）把参数装入寄存器，下面是插入到我们的系统调用中的对 `unistd.h` 的修改：

```

-----
include/asm/unistd.h
1: /*
2: * This file contains the system call numbers.
3: */
4:
5: #define __NR_restart_syscall 0
6: #define __NR_exit 1
7: #define __NR_fork 2
8: ...
9: #define __NR_utimes 271
10: #define __NR_fadvise64_64 272
11: #define __NR_vserver 273
12: #define __NR_ourcall 274
13:
14: /* #define NR_syscalls 274 this is the old value before our syscall */

```

```
15: #define NR_syscalls 275
```

最后，应当创建一个用户程序来测试这个新的系统调用，正如本节前面所述，当程序员将 C 代码中的参数加载到 x86 的寄存器中时，可以借助于系统提供的一系列宏。在 `/usr/include/asm/unistd.h` 文件中定义了七个宏：`_syscallx(type,name,...)`，其中 `x` 表示参数的个数。每个宏都会根据 0~5 的某个数字加载对应数量的参数，而 `syscall6(...)` 宏只传递一个指向更多参数的指针。以下示例程序使用一个参数。在这个例子中（在第 5 行），使用 `/unistd.h` 文件中的 `_syscall1(type,name,type1,name1)` 宏，这个宏根据适当的参数，通过 `int 0x80` 入口对一个系统调用进行解析：

```
-----
mytest.c
1: #include <stdio.h>
2: #include <stdlib.h>
3: #include "/usr/include/asm/unistd.h"
4:
5: _syscall(long,ourcall,long, num);
6:
7: main()
8: {
9: printf("our syscall --> num in=5, num out = %d\n", ourcall(5));
10: }
```

10.3 构建和调试

在内核中添加代码后就需要不断运行，修正错误。本节将描述如何调试你编写的内核代码以及如何构建与调试相关的工具。

调试设备驱动程序

在前述章节中，我们使用 `/proc` 文件系统来搜集有关内核的信息。我们同样可以通过 `/proc` 文件系统来获得与设备驱动程序有关的信息，以便于用户访问，而且是调试设备驱动程序的好方法。当对 `/proc` 文件系统进行读写操作时，它的每个节点都连接到一个内核函数。在 Linux 2.6 内核中，对内核部分（包括设备）的大多数写操作都是通过 `sysfs` 而不是 `/proc` 文件系统来完成的。在内核运行的过程中，这些操作修改某个具体内核对象的属性。`/proc` 文件系统在只读操作中仍然非常有用，不过这些只读操作需要处理大量数据而不太关心对象的属性——值对。本节只讨论从 `/proc` 文件系统中读入数据的情况。

要想对你的设备进行读访问，首先就要在 `/proc` 文件系统中创建一个节点^①，这可以通过函数 `create_proc_read_entry()` 来实现：

```
-----
include/linux/proc_fs.h
```

① 每一个节点在整个目录树中，是一个文件或是一个目录。——译者注

```

146 static inline struct proc_dir_entry *create_proc_read_entry(const char
    *name,
147     mode_t mode, struct proc_dir_entry *base,
148     read_proc_t *read_proc, void * data)
-----

```

*name 是在 /proc 文件系统中出现的节点项，如果 mode 等于 0，表示任何进程都可以读取该文件。如果为单个设备驱动程序创建了多个不同的 proc 文件，最好是先使用函数 proc_mkdir() 建立一个 proc 目录，然后把所有 proc 文件都放到该目录下。*base 是 /proc 下放置 proc 文件的目录路径，如果其值为 NULL，表示文件就直接在 /proc 目录下。读取该文件时，可以调用 *read_proc 函数，*data 则是一个被传回给 *read_proc 的指针：

```

-----
include/linux/proc_fs.h
44 typedef int (read_proc_t)(char *page, char **start, off_t off,
45     int count, int *eof, void *data);
-----

```

这是可通过 /proc 文件系统读取的函数原型。*page 是一个指向缓冲区的指针，该函数会向这个缓冲区写入数据，以便进程读取 /proc 文件的内容。该函数必须从 *page 所指缓冲区中偏移为 off 个字节的地方开始写数据，并且写的数据不能超过 count 个字节。由于绝大多数读操作仅返回少量信息，因此很多实现都忽略了 off 和 count。此外，**start 指针在内核中也极少用到，也可以不考虑它。但如果你实现的读函数返回大量数据，那就要使用 off、count 以及 **start 来管理一次只读入少量信息的数据块。当读操作完成后，该函数应当向 *eof 写入 1，而 *data 将作为参数，传递给在 create_proc_read_entry() 函数中定义的读函数。

10.4 小结

本章覆盖了设备驱动程序、模块以及系统调用等内容，并描述了 Linux 使用设备驱动程序的不同方式。

更准确地说，我们探讨了如下几个主题。

- 描述了 Linux 文件系统中的 /dev 树，并论述了如何确定什么样的设备驱动程序控制什么样的设备。
- 解释了设备驱动程序如何利用文件数据结构和文件操作结构来处理文件系统的 I/O 操作。
- 讨论了用户级内存与内核内存的不同之处，以及设备驱动程序如何在两者之间传递数据。
- 分析了 Linux 内核中等待队列的建立过程，并举例说明当设备驱动程序需要等待特定资源时应该如何使用等待队列。
- 探索了等待队列和中断的原理，这也是需要将 CPU 的控制权交给其他进程时，Linux 内核用来明确中断设备驱动程序处理过程的方法。
- 介绍了 Linux 的系统调用，并概述其基本功能。
- 讨论了块设备驱动程序与字符设备驱动程序的不同之处，并介绍了 linux 2.6 中引入的新设备模型，同时对 sysfs 进行了简要说明。

第 10 章的第一部分，我们抽象地介绍了以上主题，并从以上各个方面分析了特殊的设备驱动

程序/dev/random。第二部分通过具体的例子和示例代码,说明如何构造一个实际的设备驱动程序。更准确地说,可以归纳为以下几点。

- 介绍了如何在/dev目录下创建一个与设备驱动程序关联的节点,以及如何创建动态模块。
- 介绍了 Linux 2.6 中从设备驱动程序模块输出符号的新方法。
- 举例说明设备驱动程序如何提供 IOCTL 函数,这些函数允许设备通过文件系统与内核交互。
- 解释了中断和轮询如何产生,以及自旋锁在 x86 及 PPC 结构体系中的不同。
- 介绍了如何向 Linux 内核添加一个简单的系统调用。

第 10 章为我们在 linux 2.6 版本中开发设备驱动程序打下了坚实的基础,实际上,本书前面的章节也介绍过这些想法和概念。

10.5 习题

1. 阅读第 3 章关于编译内核和用户代码部分,并重新编译内核及 mytest.c 文件,然后运行 mytest.c 文件并观察其输出结果。
2. 给 ourcall 添加一个参数。
3. 在 ourcall 中创建一个系统调用。
4. 解释系统调用与设备驱动程序之间的异同。
5. 为什么不能使用 memcpy 在用户空间和内核空间之间复制数据?
6. 上半部中断处理例程和下半部中断处理例程的区别是什么?
7. tasklet 与 work_queue 有何区别?
8. 当一个设备不仅仅能处理简单的读写请求时, Linux 如何与它进行交互?
9. 未锁定的自旋锁在 x86 平台上的数值是多少? 在 PPC 平台上呢?
10. 请用一句话描述块设备驱动程序与字符设备驱动程序的不同之处。



[G e n e r a l I n f o r m a t i o n]

书名= L i n u x内核编程

作者= (美) 罗德里格斯, (美) 费舍尔, (美) 斯莫斯基著

页码= 4 0 0

I S B N = 4 0 0

S S 号= 1 2 8 1 6 7 9 4

d x N u m b e r = 0 0 0 0 0 8 1 0 2 6 5 2

出版时间= 2 0 1 1 . 0 5

出版社= 该引擎未能查询到

定价: 7 5 . 0 0

试读地址= <http://book.szdn.net.org.cn/bookDetail.jsp?dxNumber=000008102652&d=2FD8B2B744FE27D2F19554E667C0290F&fenlei=18170403060806&sw=%B1%E0%B3%CC>

全文地址= <http://Wqa.5read.com/image/ss2jpg.dll?did=b1&pid=089F02215B594548D94AAD2DCE8268A706F9BD38CA8B0C4EBF2DC62CA7485254B4F94D81BF0EA2EB25F4AC83F0E02DC46E038E7331C55B47CC75D298411D7152433866735C288A553FE010A65A58A16DD2C57B9C7DA76B888FF5C39FC1CA61438C7C75EAA8440DC7A72B5A91A6F276CD4BE9&jid=/>